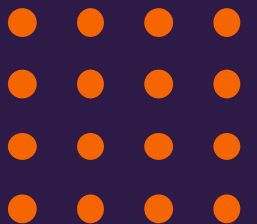
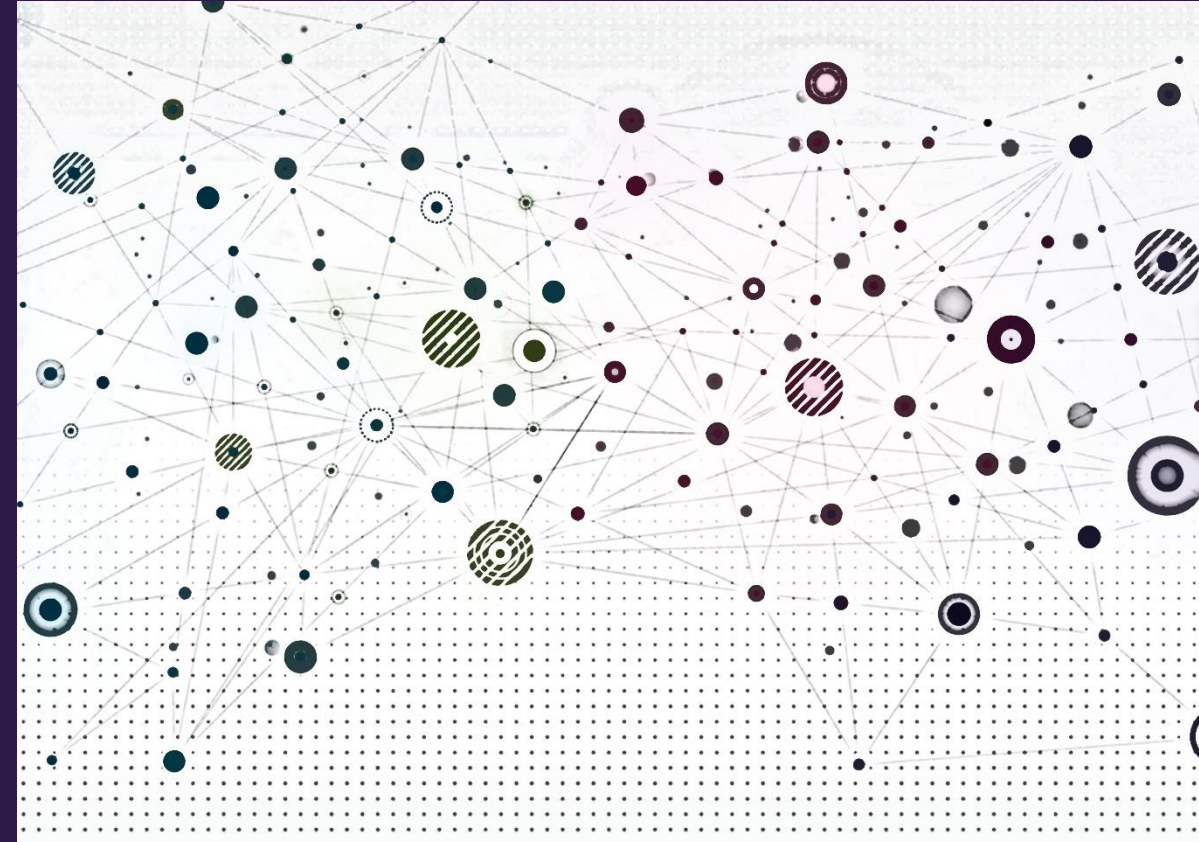




GNNs

Theoretical Perspective





Overview

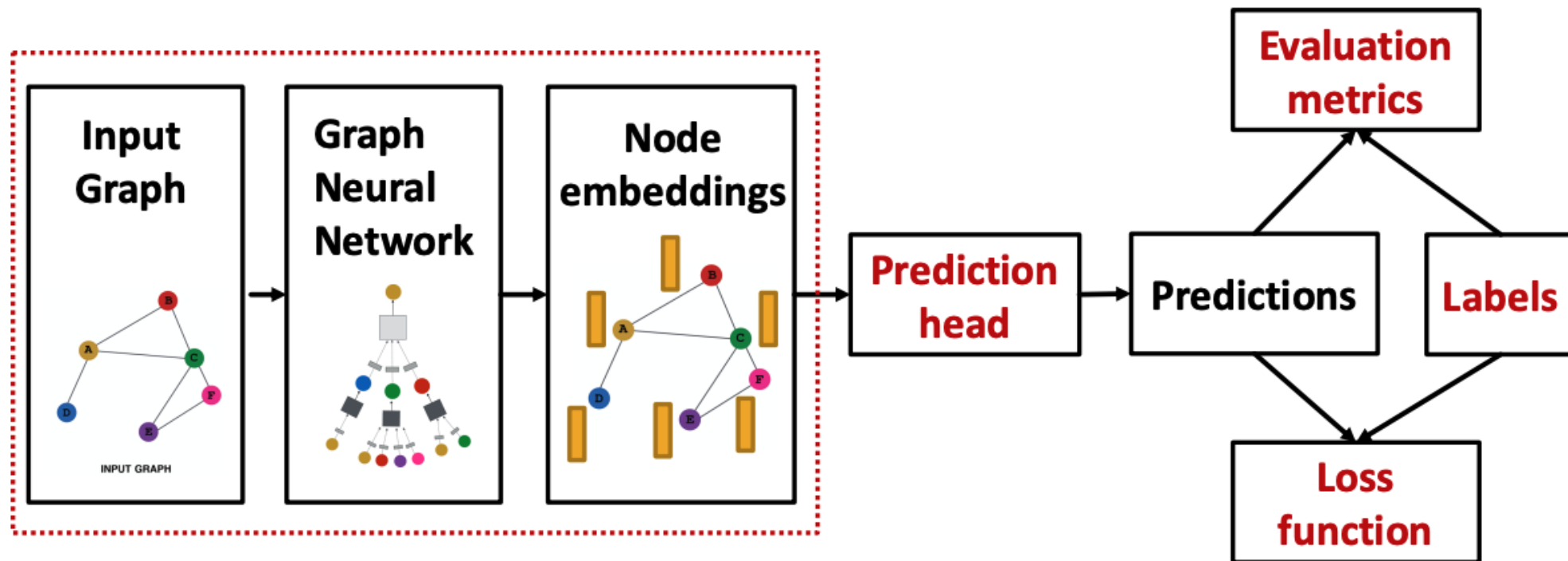
- Recap
- GNN computation graphs
- Theoretical analysis of GNN aggregation functions
- Designing an injective GNN



Recap



GNN training pipeline



Information modalities in graphs

Technological networks

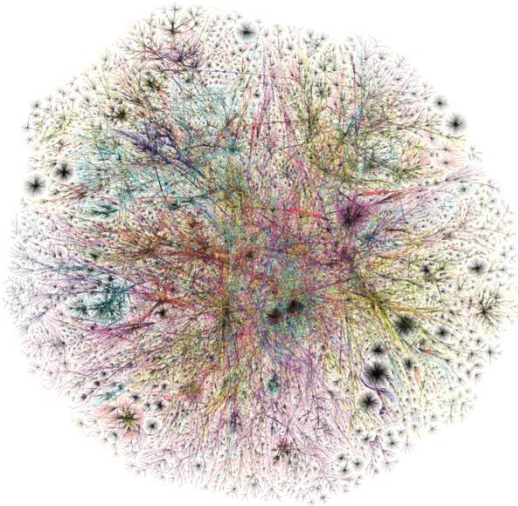


Image credit: Newman 2018

Information networks

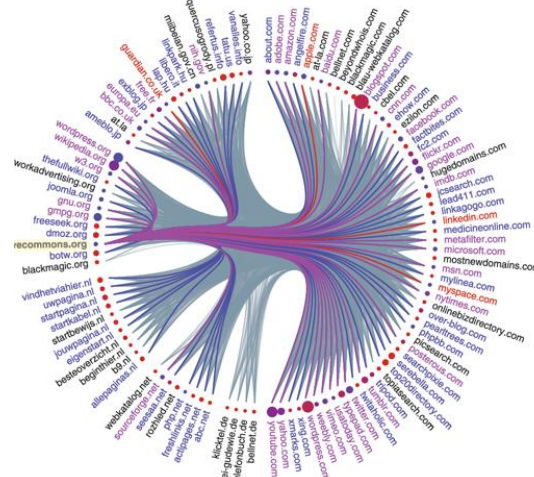


Image credit: [American Scientist](#)

Biological networks

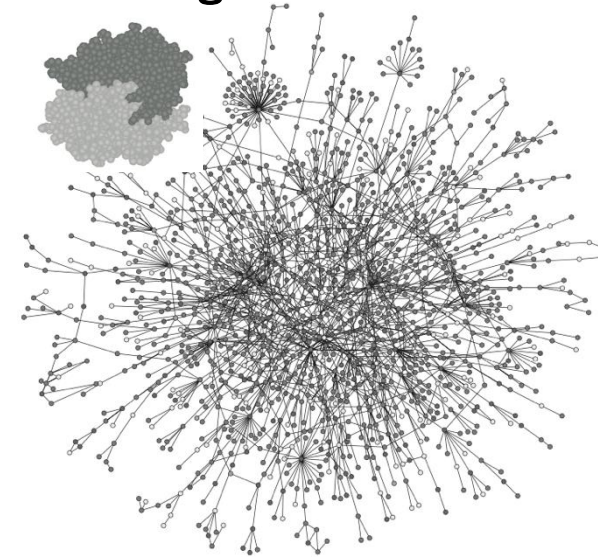
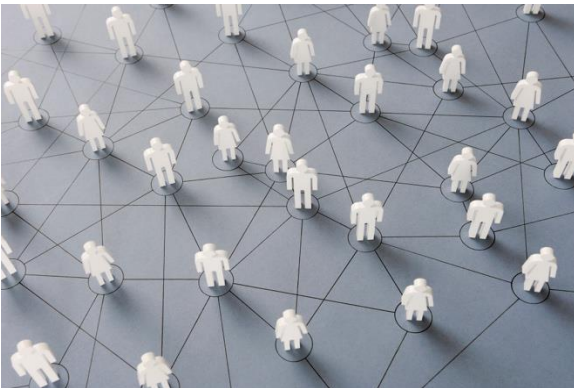


Image credit: Newman 2018

Social networks



Many more ...

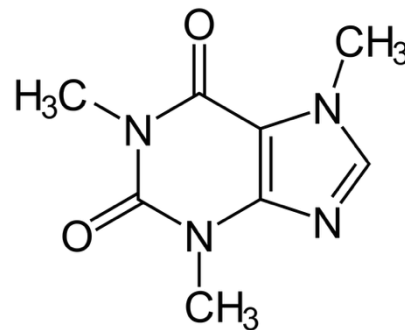


Image credit: [Chemistry StackExchange](#)

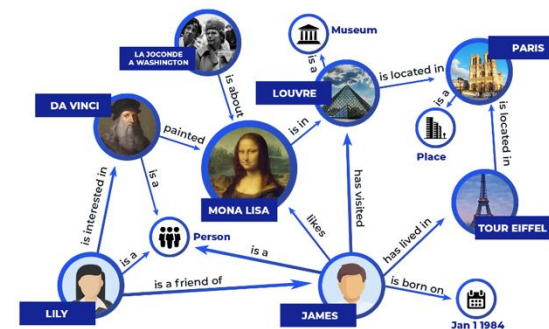
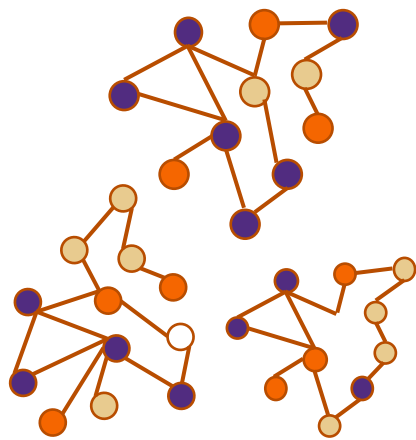


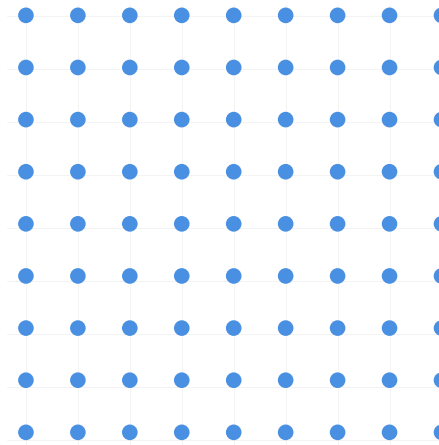
Image credit: [Zilliz](#)

Why is graph deep learning hard?

- Graphs have complex topological structure (i.e., irregular arrangement, no spatial locality like grids)
- No fixed ordering or reference points



Graph Data



Images



Text



Theory of GNNs

- How expressive are GNNs?
 - Many GNN models have been proposed (e.g., GCN, GAT, GraphSAGE, ...)
 - Last count, **67** different types of convolutional layers in PyG
 - How well can these GNN models distinguish between different graph structures?
 - How can we design a maximally expressive GNN model?



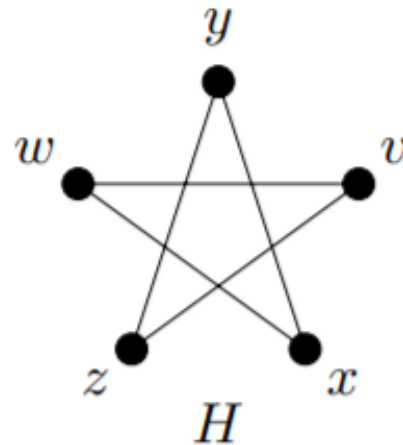
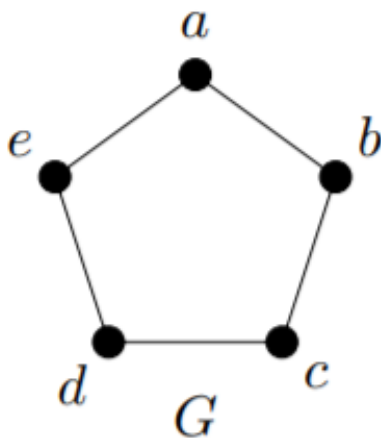
GNN computation graphs





Graph isomorphism

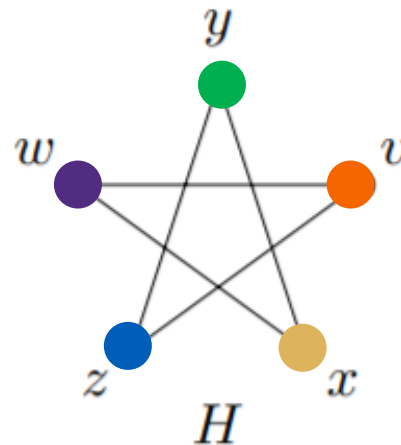
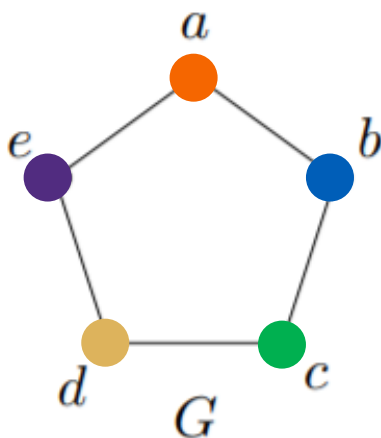
- Consider two graphs, G_1 and G_2
- An **isomorphism**, ψ , is a one-to-one mapping between the vertices in G_1 to the vertices in G_2 such that any two nodes u and v are connected in G_1 if and only if $\psi(u)$ and $\psi(v)$ are connected in G_2





Graph isomorphism

- Consider two graphs, G_1 and G_2
- An **isomorphism**, ψ , is a one-to-one mapping between the vertices in G_1 to the vertices in G_2 such that any two nodes u and v are connected in G_1 if and only if $\psi(u)$ and $\psi(v)$ are connected in G_2

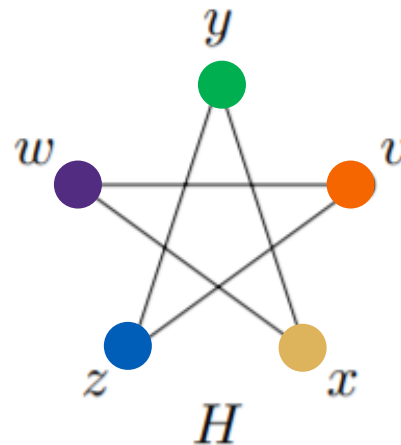
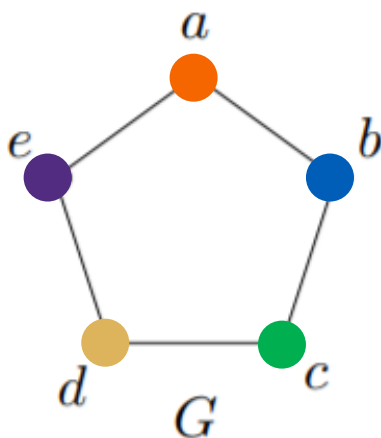


$\varphi(a) = v;$
 $\varphi(b) = z;$
 $\varphi(c) = y;$
 $\varphi(d) = x;$
 $\varphi(e) = w$



Graph isomorphism

- Consider two graphs, G_1 and G_2
- An **isomorphism**, ψ , is a one-to-one mapping between the vertices in G_1 to the vertices in G_2 such that any two nodes u and v are connected in G_1 if and only if $\psi(u)$ and $\psi(v)$ are connected in G_2
- If an isomorphism exists between two graphs, **the graphs are called isomorphic**
- No (yet) polynomial time algorithm to determine graph isomorphism

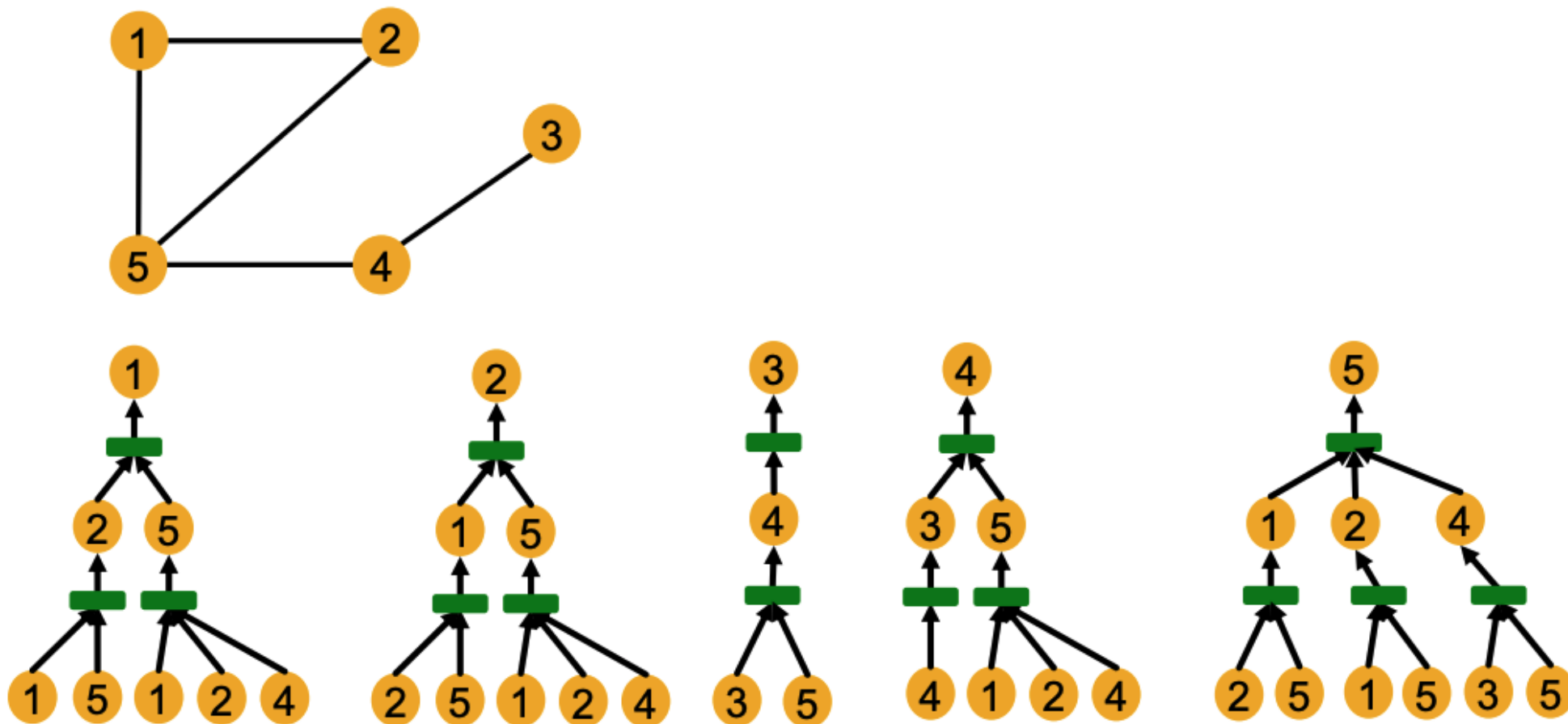


$$\begin{aligned}\varphi(a) &= v; \\ \varphi(b) &= z; \\ \varphi(c) &= y; \\ \varphi(d) &= x; \\ \varphi(e) &= w\end{aligned}$$



A simpler problem

Given a pair of nodes with different neighborhood structures, is there a GNN that can always differentiate them?





Single GNN layer

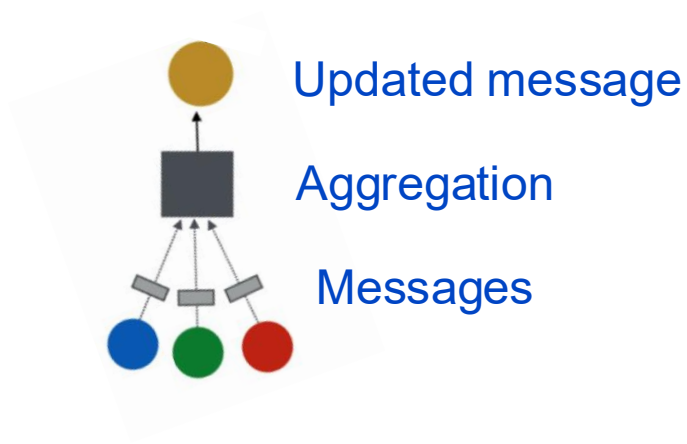
- Let's consider message passing GNNs:

- Aggregation:** each node aggregates messages (embeddings) of neighbor nodes

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \mathbf{h}_u^{(k)}, \forall u \in N(v) \right\} \right)$$

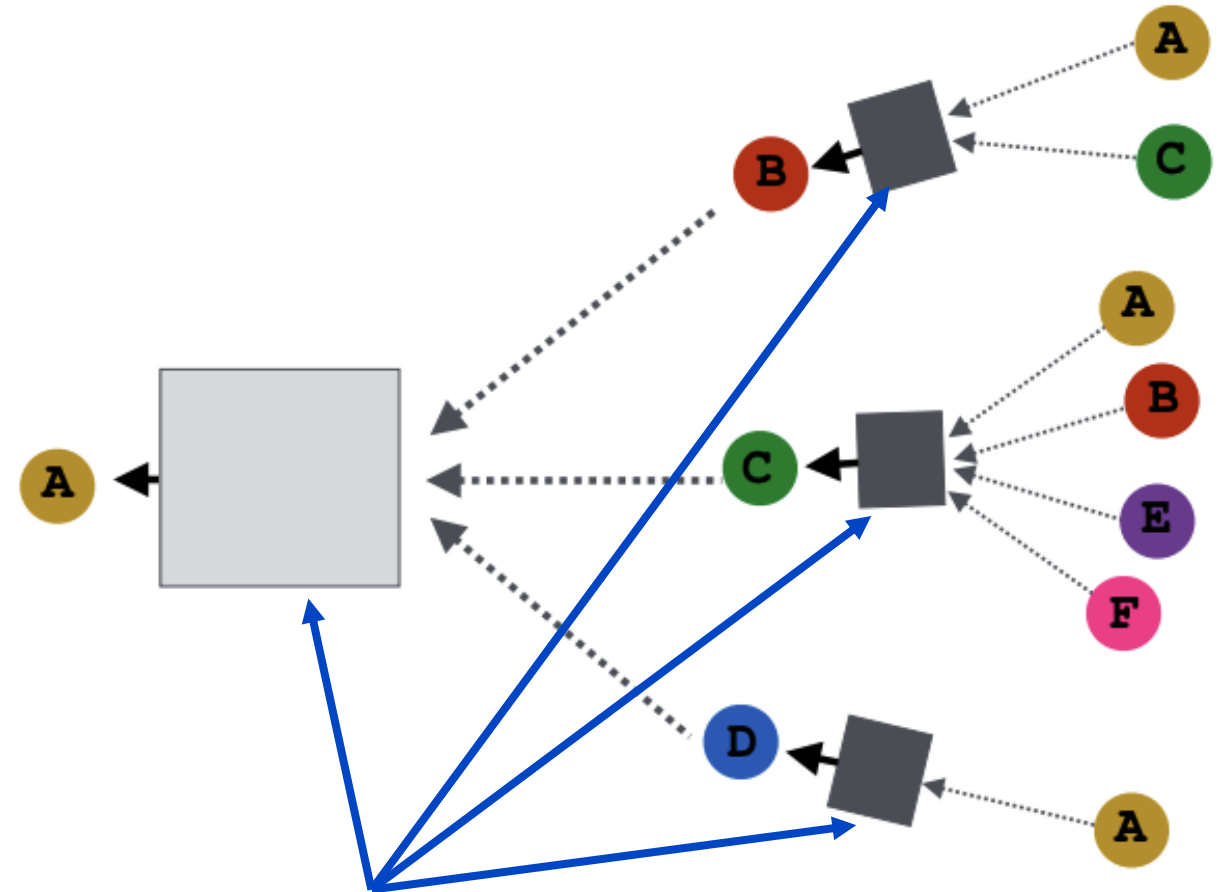
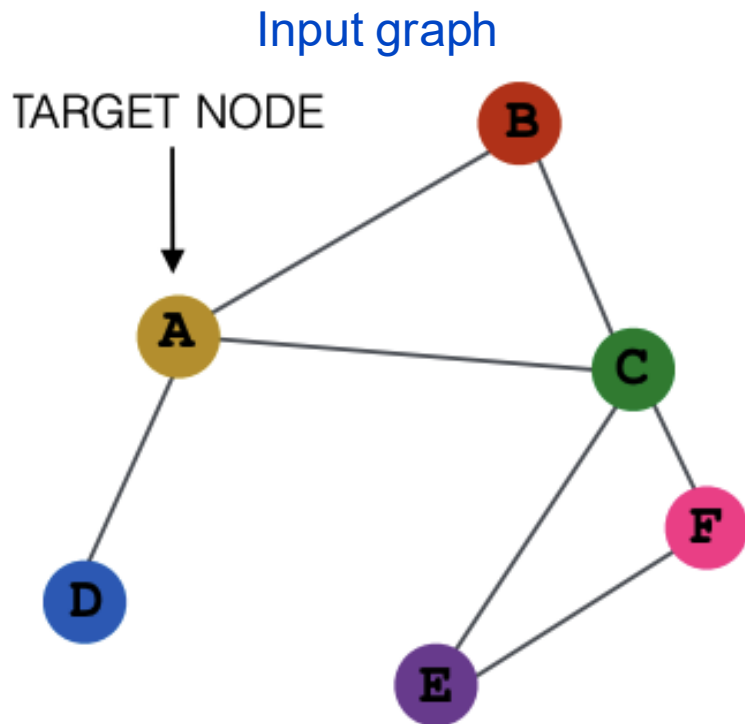
- Update:** each node updates its embedding (message for next layer)

$$\mathbf{h}_v^{(k+1)} = \text{Update}^{(k)} \left(\mathbf{h}_v^{(k)}, \mathbf{m}_{N(v)}^{(k)} \right)$$





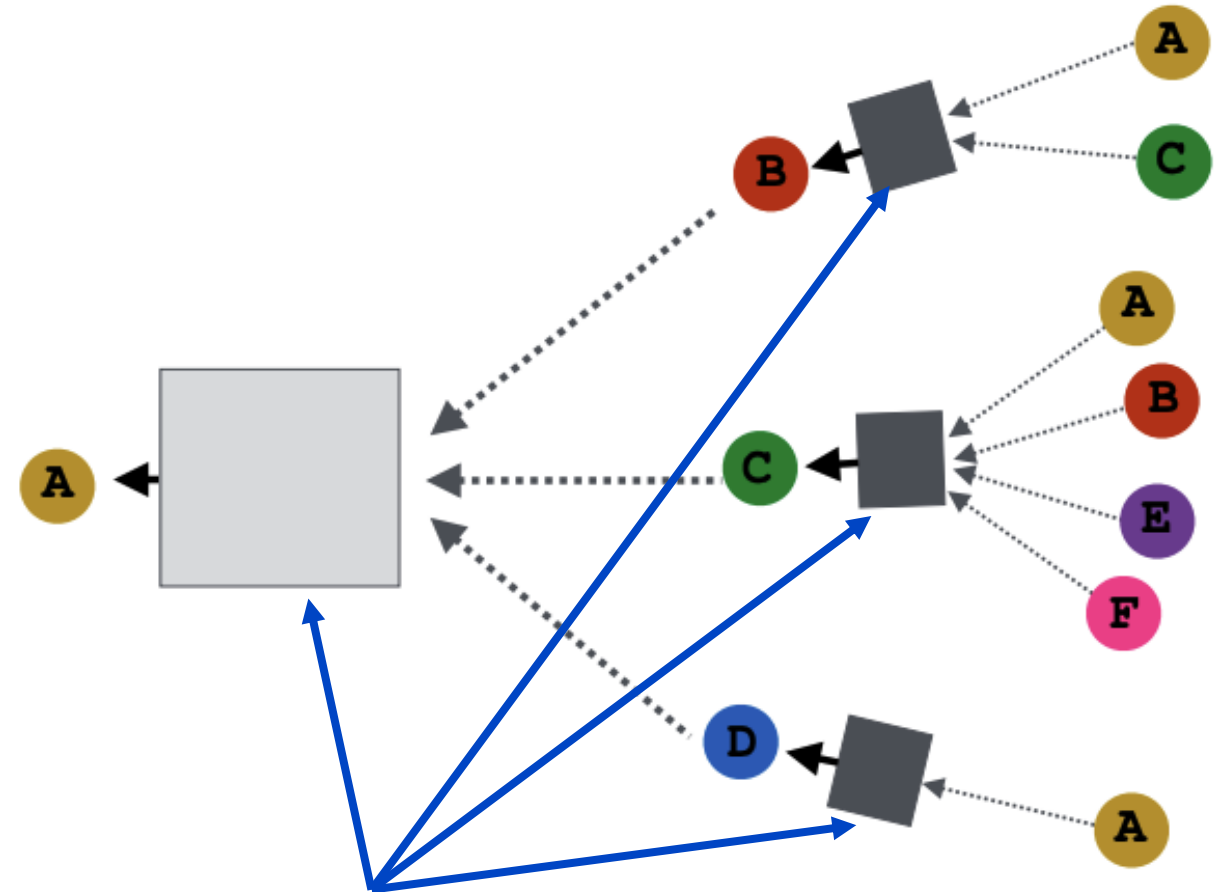
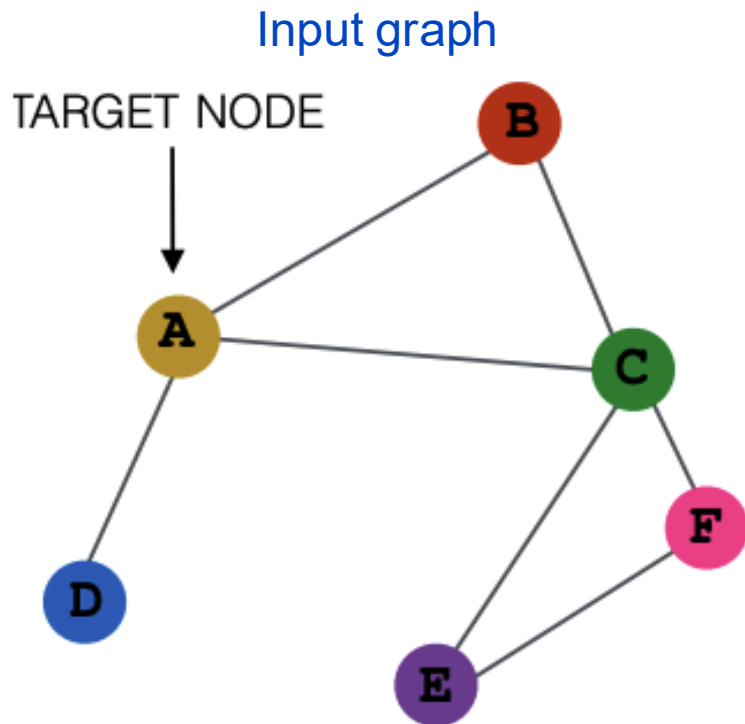
Many message passing GNN layers are possible



Different GNN models (GCN, GAT, etc) use different neural networks



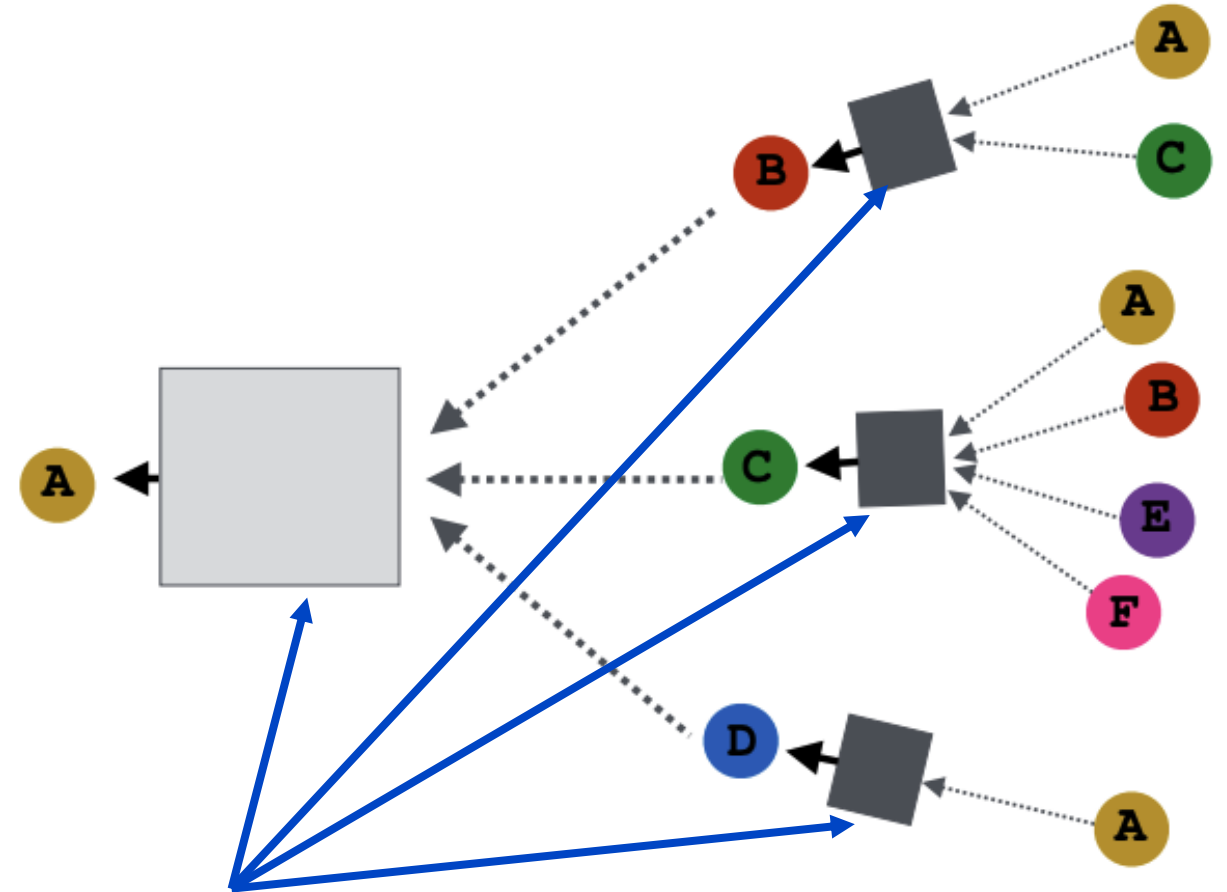
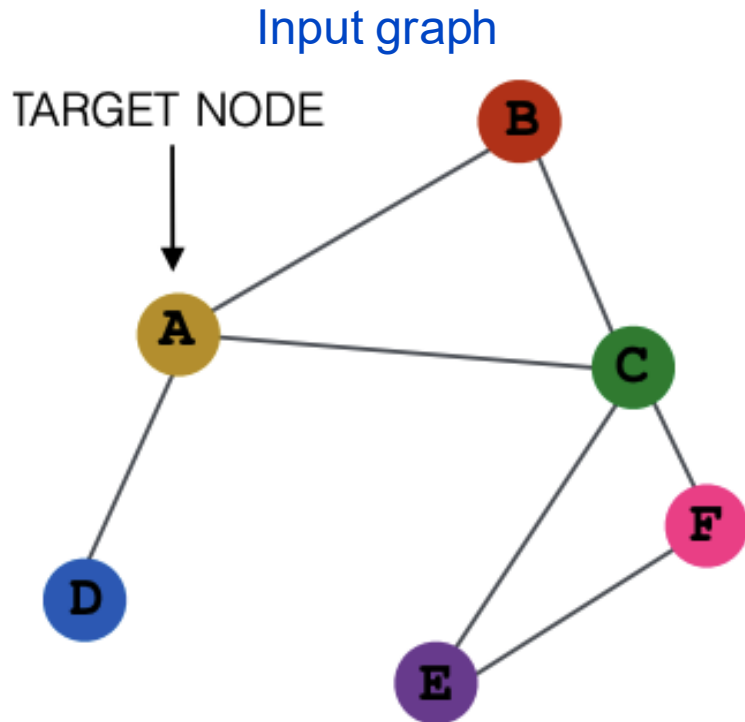
GCN with global mean pooling



Element wise mean pooling (agg) +
linear and ReLU (update)



GraphSAGE with global max pooling

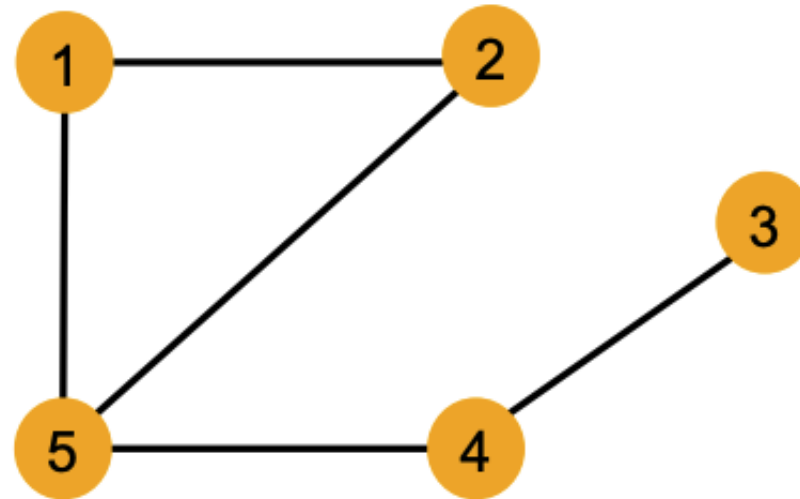


Neighbor sample and element wise max pooling (agg) + MLP (update)



Can a GNN distinguish graph structures

- Use node **colors** to indicate nodes with the same feature values (i.e., two nodes with the **same color have the same input feature vector**)
- For example, the graph here assumes all nodes have the same features

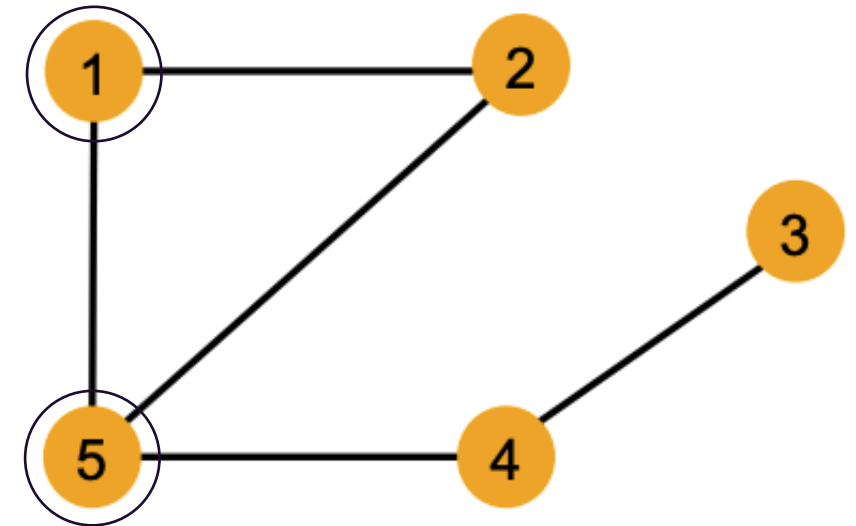


- Question: Can a GNN always distinguish these nodes? IOW, will the embeddings be different for every node?



Local neighborhood structure

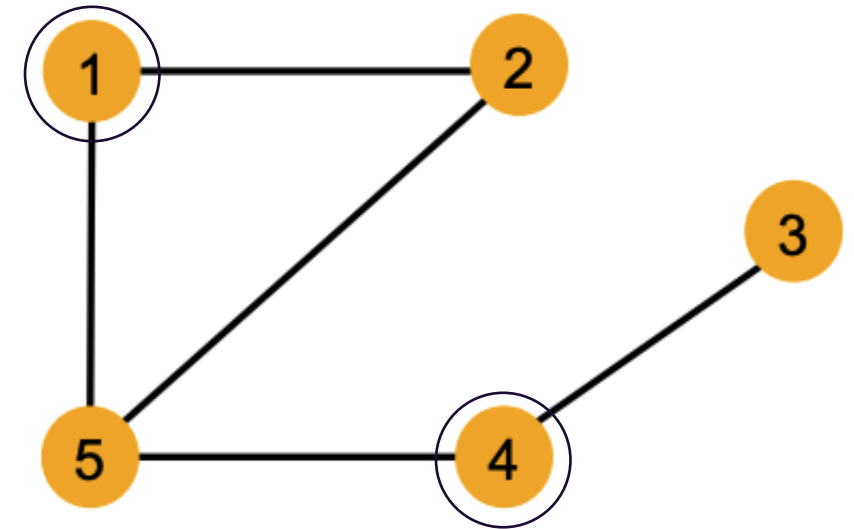
- Consider the **local neighborhood structure** around each node in a graph
 - Example: **Nodes 1 and 5** have **different** neighborhood structures indicated by the fact that they have different degrees





Local neighborhood structure

- Consider the **local neighborhood structure** around each node in a graph
 - Example: **Nodes 1 and 4** have the same degree 2. However, they have **different** neighborhood structures because **their neighbors have different node degrees**

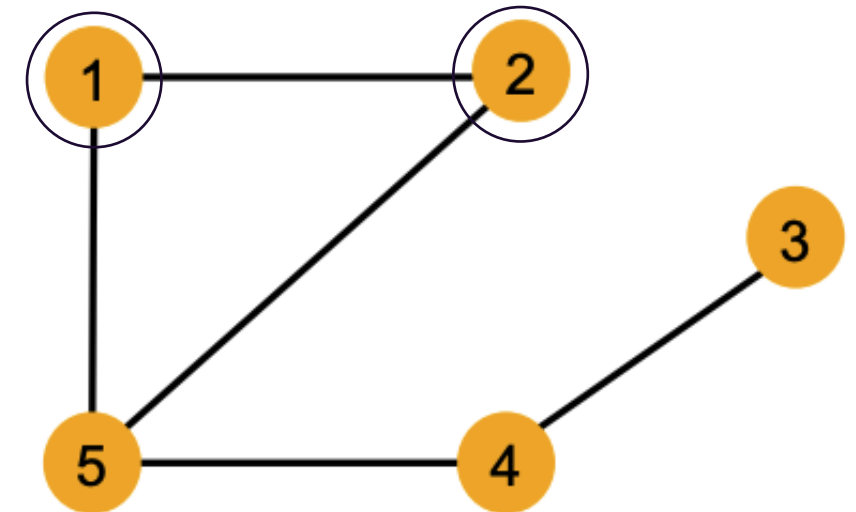


Node 1 has neighbors with degrees 2 and 3
Node 4 has neighbors with degrees 1 and 3



Local neighborhood structure

- Consider the **local neighborhood structure** around each node in a graph
 - Example: **Nodes 1 and 2** have the same degree neighborhood structure because **they are symmetric within the graph**

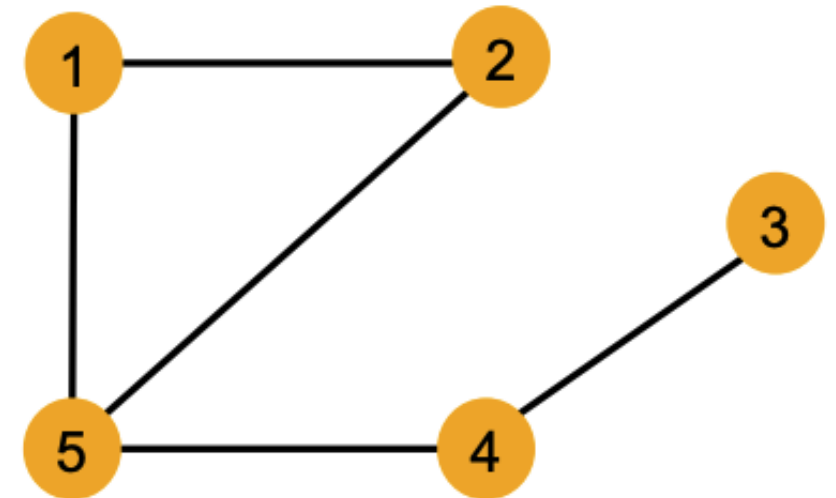


Node 1 has neighbors with degrees 2 and 3
 Node 2 has neighbors with degrees 2 and 3
 Their **second hop neighbors** also have the same degrees (node 4 with degree 2)



Local neighborhood structure

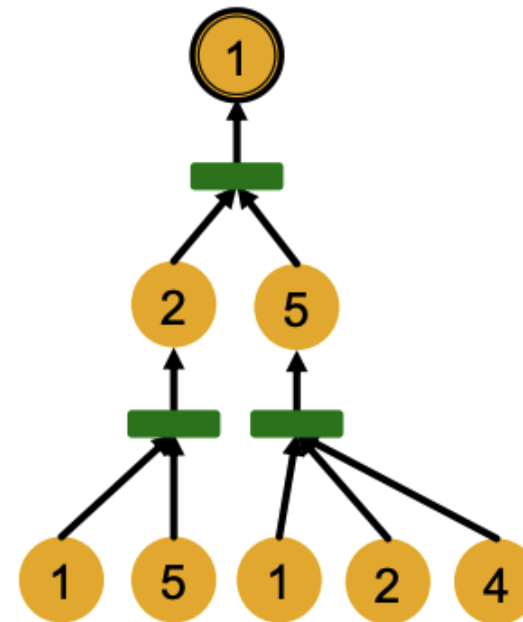
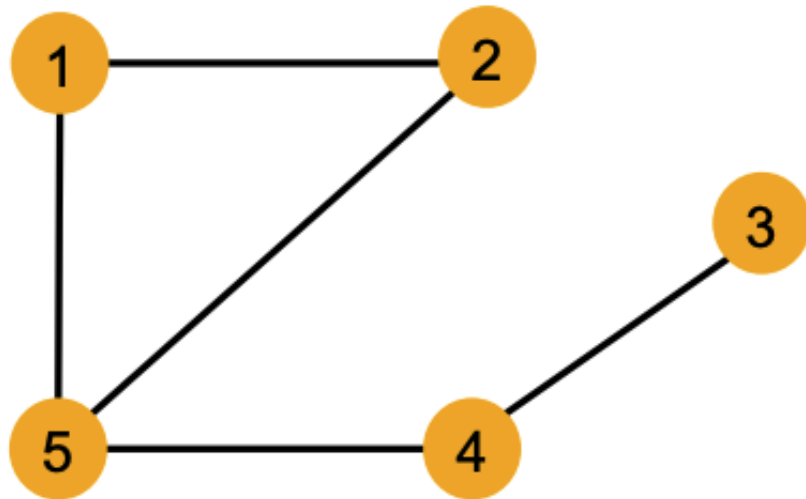
- **Question:** Can GNN node embeddings distinguish different nodes' local neighborhood structures?
 - Under what conditions do they succeed or fail?
- Need to understand how local neighborhood structure determines GNN node embedding **computation graph**





Computation graphs in a GNN

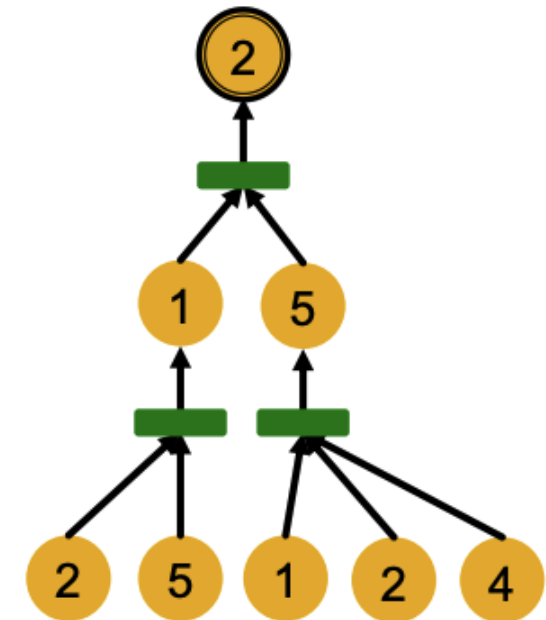
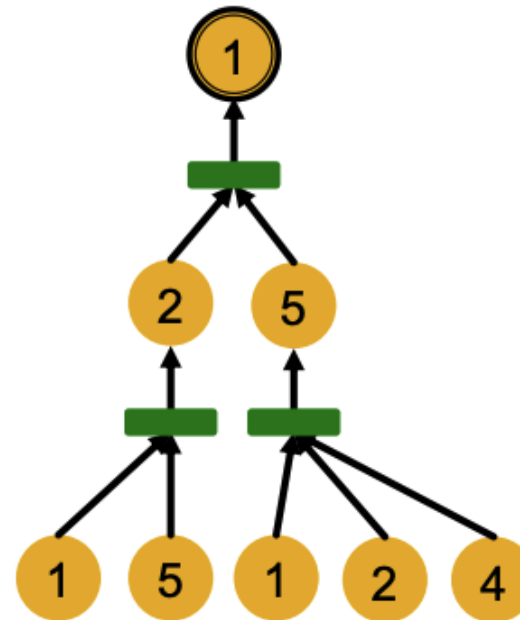
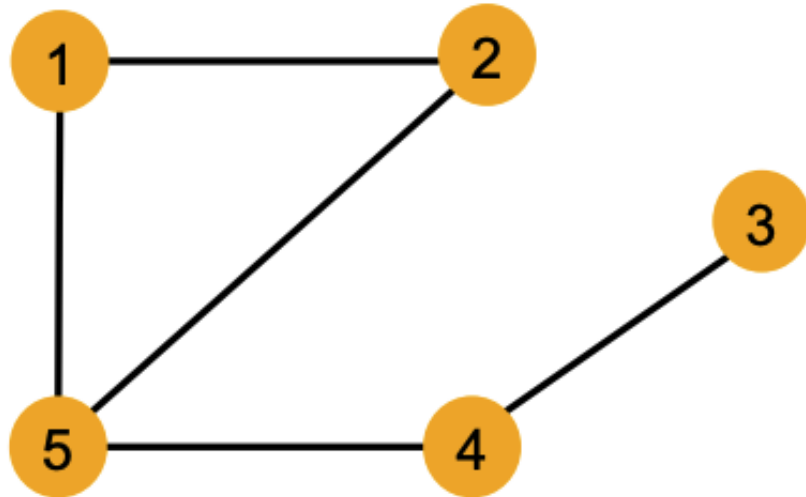
- GNN layers generate node embeddings through a computational graph determined by the node neighborhood
- Example: Computation graph for **node 1**





Computation graphs in a GNN

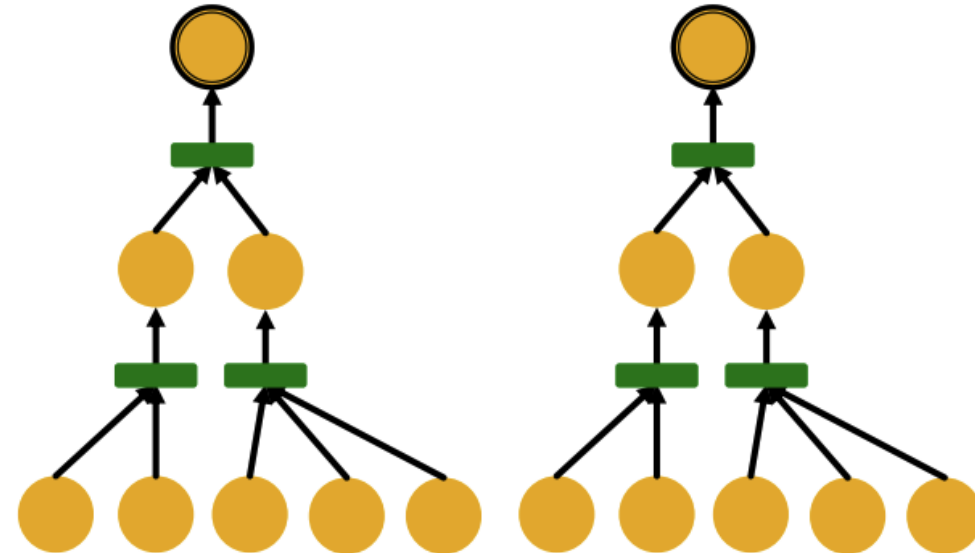
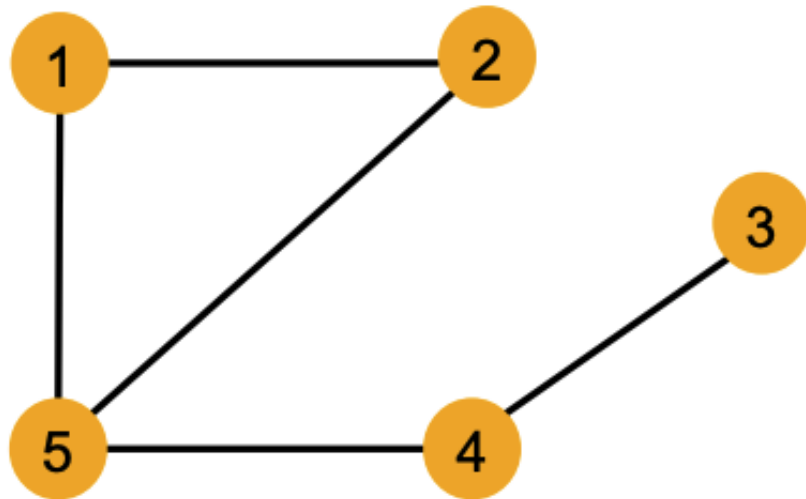
- Compare computation graph for node 1 and node 2





Computation graphs in a GNN

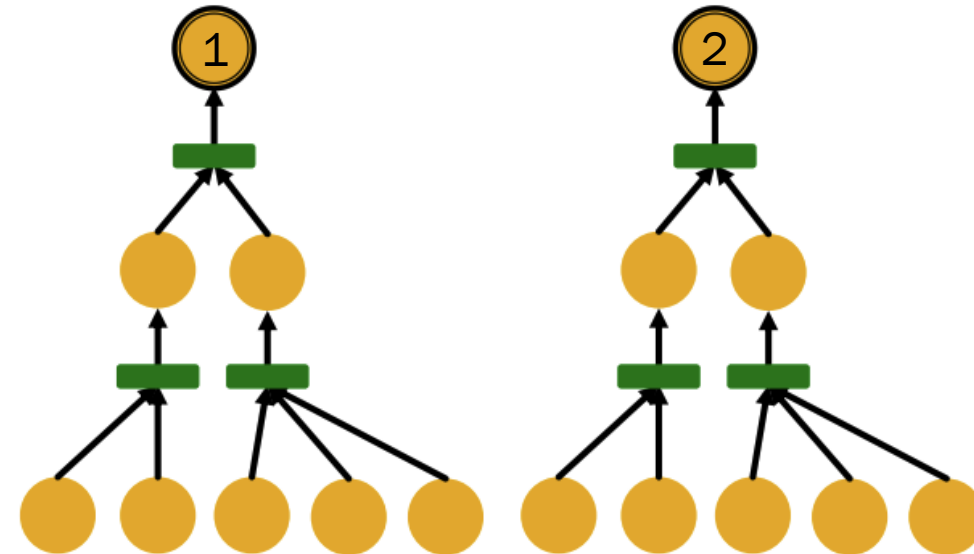
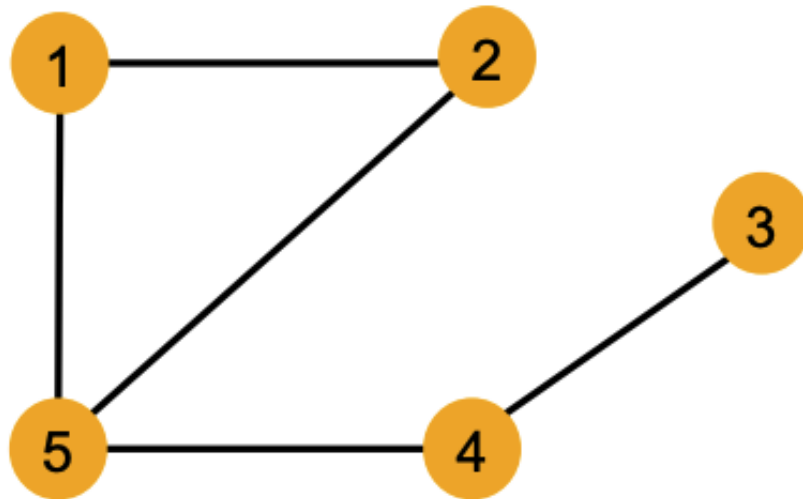
- Compare computation graph for **node 1** and **node 2**
- The GNN only sees node features (not IDs):





Computation graphs in a GNN

- Any message passing GNN with shared weights will generate the same embedding for nodes 1 and 2 because:
 - Computation graphs are identical
 - Node features (colors) are identical



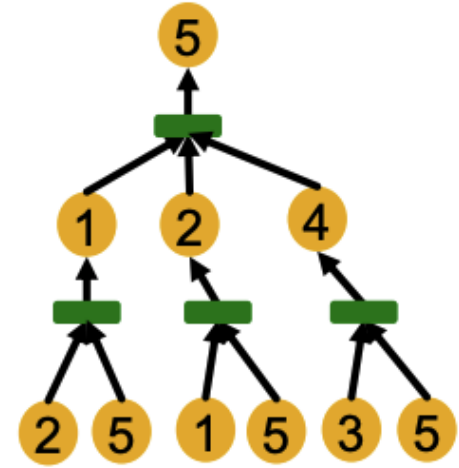
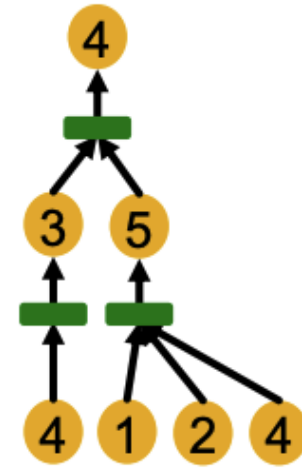
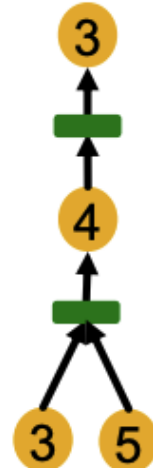
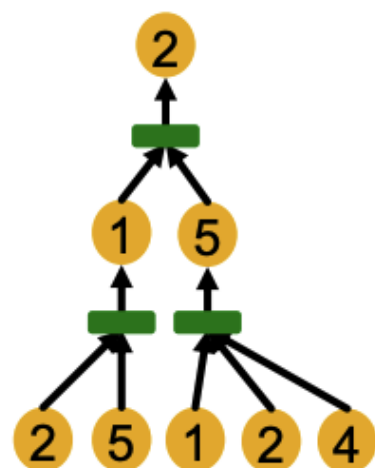
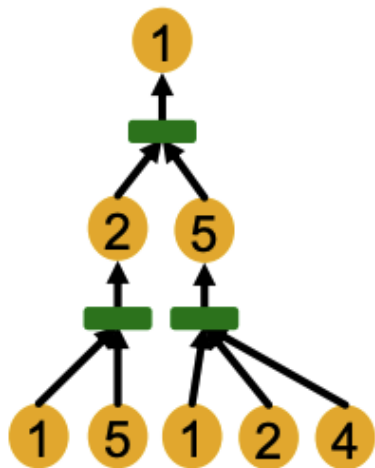
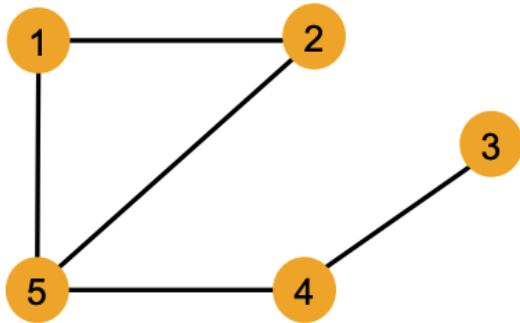
Input to GNN computation does not include node IDs

GNN will not be able to distinguish nodes 1 and 2



Computation graphs in a GNN

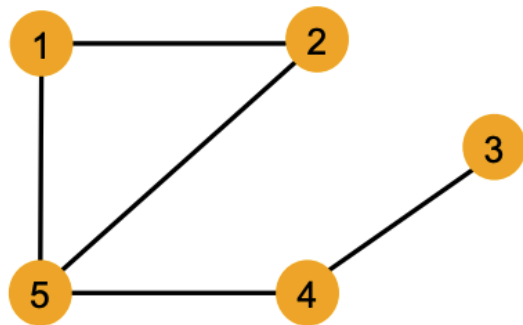
- Local neighborhoods with different structures define different computation graphs



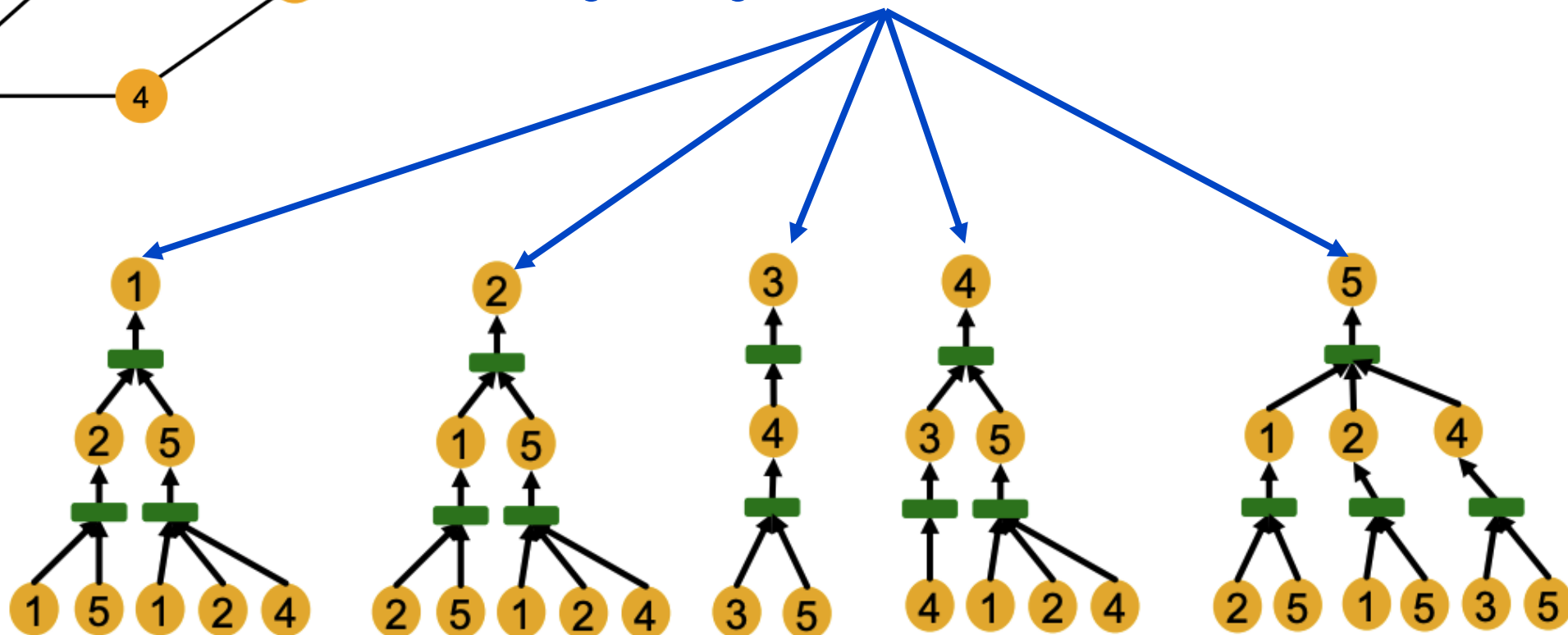


Computation graphs in a GNN

- Computational graphs for each node are **rooted trees** where the node is the root



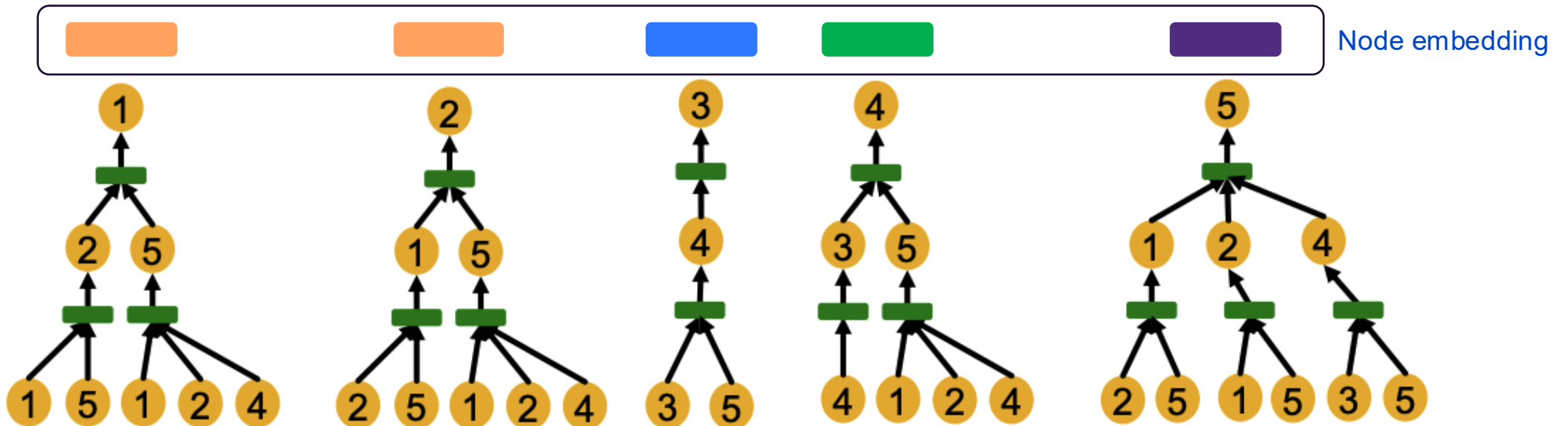
Rooted tree structures
(defined by recursively unfolding
neighboring nodes from the root node)





Computation graphs in a GNN

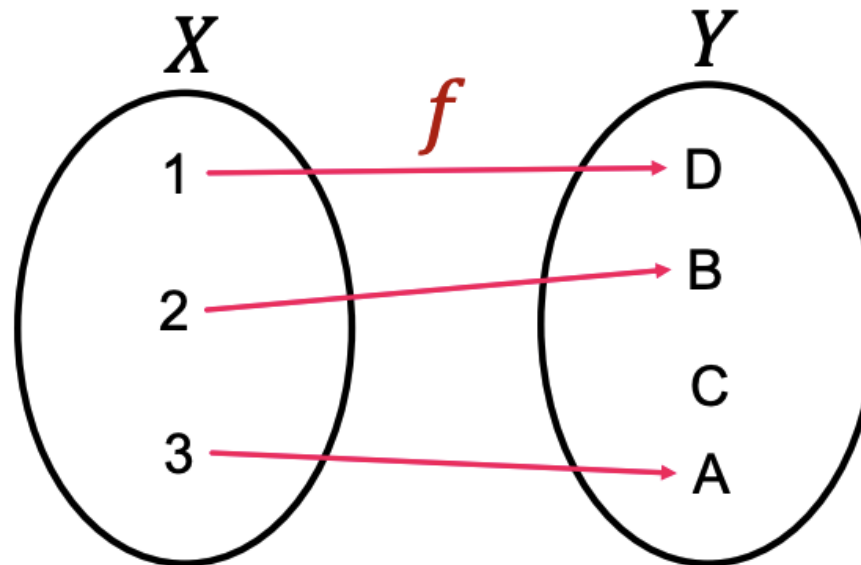
- Computational graphs for each node are **rooted trees** where the node is the root
- The most expressive GNN should **map different rooted trees to different node embeddings** (represented below by different colors)





Injective function

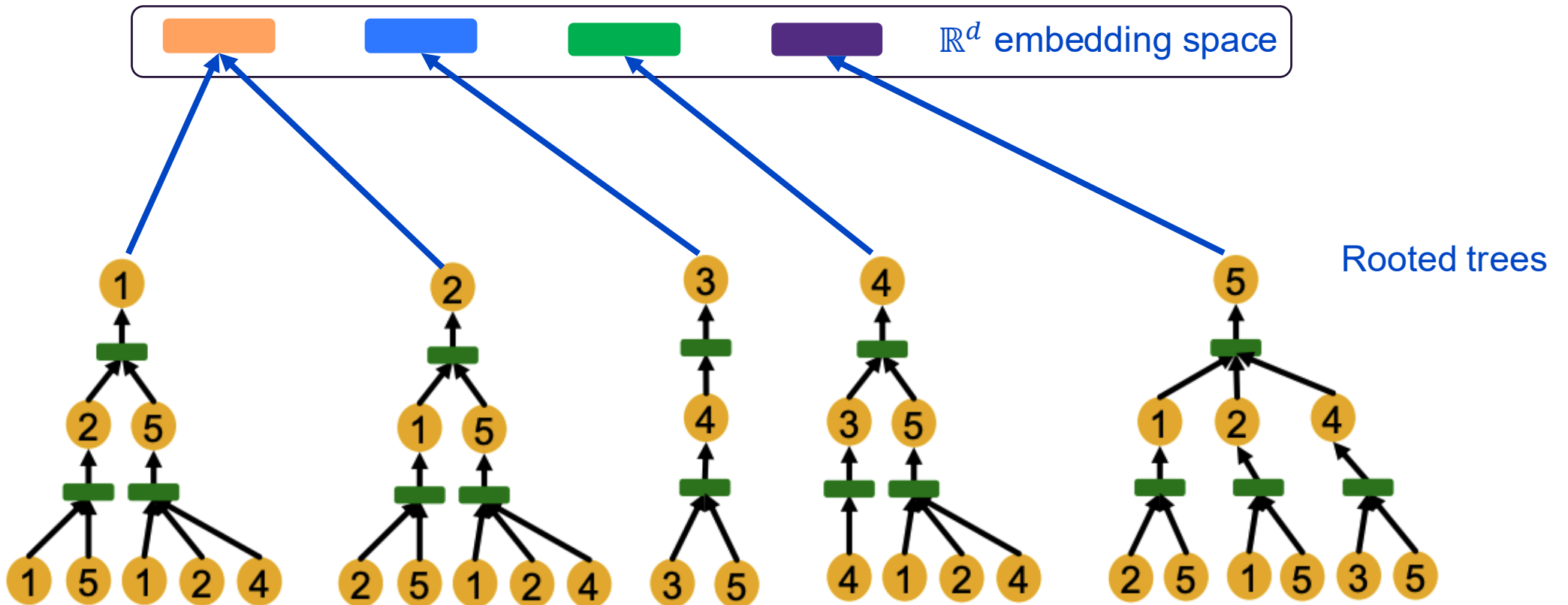
- The function $f: x \rightarrow y$ is **injective** if it maps distinct inputs, x to distinct elements y , i.e. $f(x_1) = f(x_2)$ implies $x_1 = x_2$
 - Injective: $f(x) = x^3$
 - Not injective: $f(x) = x^2$
- Conceptually, f retains all the distinguishing information about the input





How expressive are GNNs?

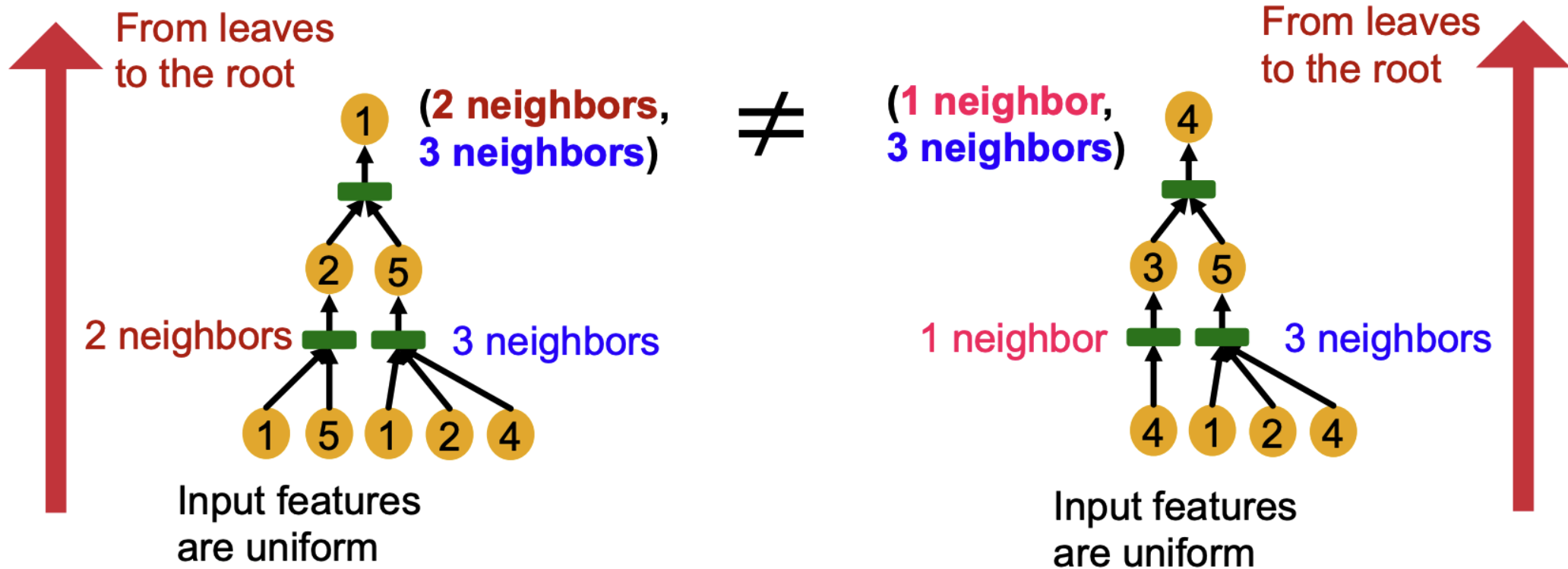
- The most expressive GNN should map rooted trees to node embeddings **injectively**





How expressive are GNNs?

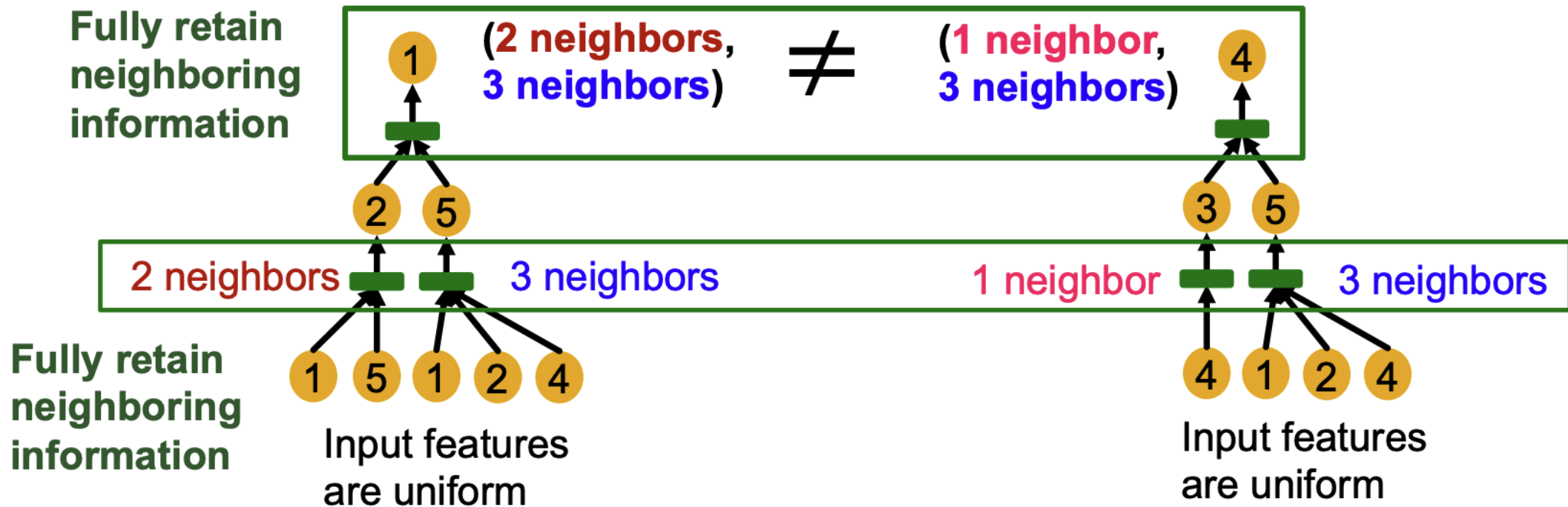
Consider that rooted trees of the same depth can be recursively characterized from the leaf nodes to the root nodes





How expressive are GNNs?

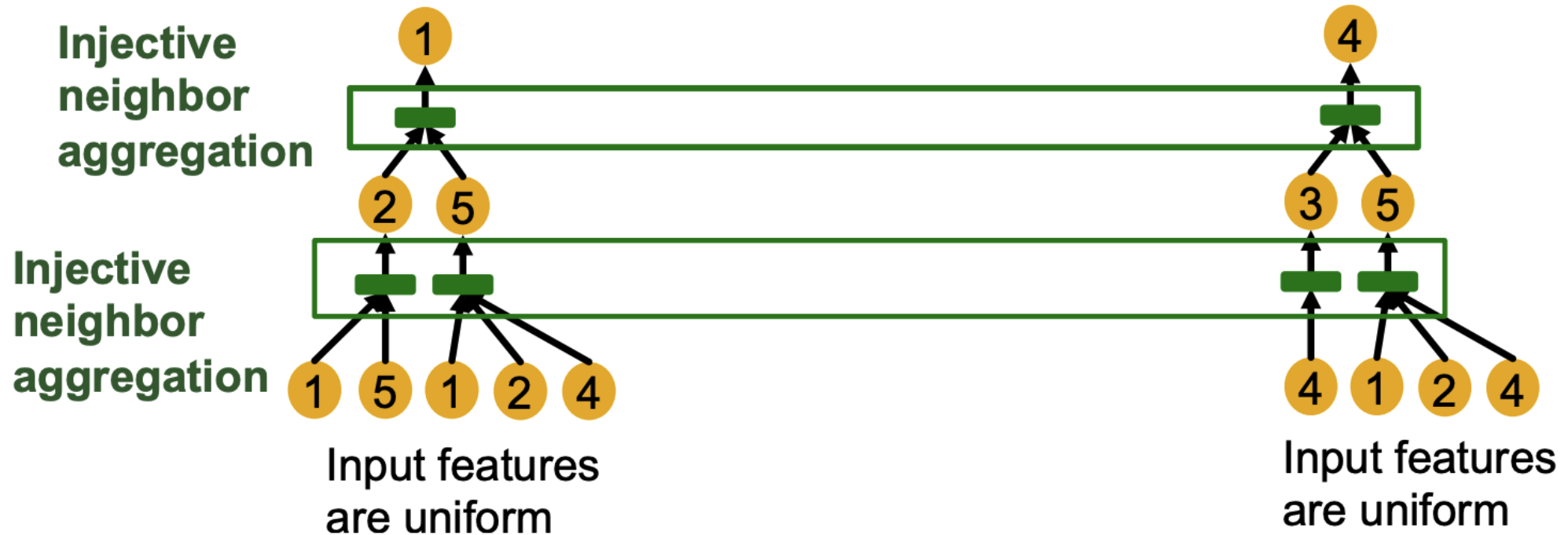
If each step of the GNN **can fully retain the neighboring information**, the node embeddings will be different for different rooted trees (i.e., for different computational graphs)





How expressive are GNNs?

Most expressive GNN must use an **injective neighbor aggregation** and update – though we focus on the aggregation (the update is easier)





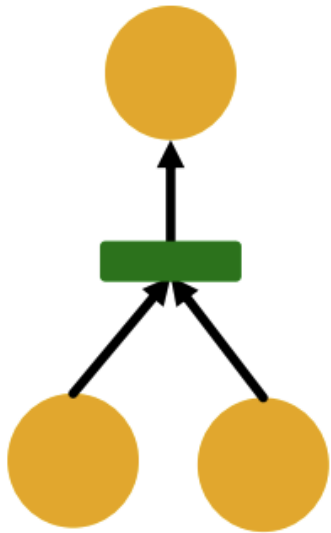
Theoretical analysis of GNN aggregation functions





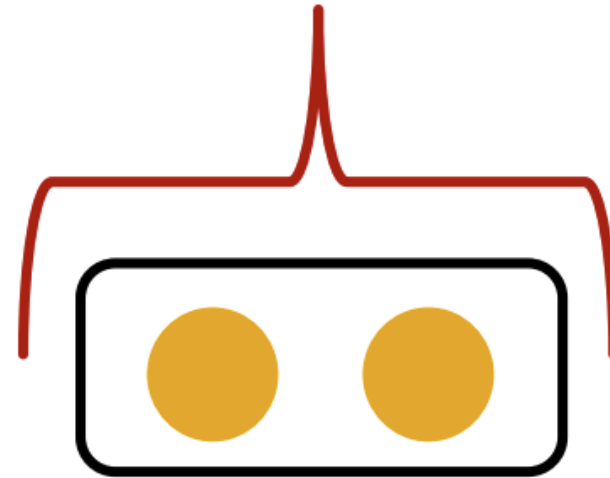
Neighbor aggregation as a function of multisets

Neighbor node aggregation can be viewed as a function of a multi-set (a set with repeating elements, equivalent to an unordered list)



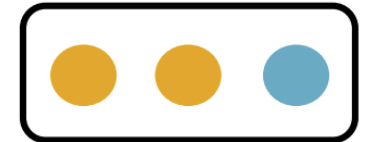
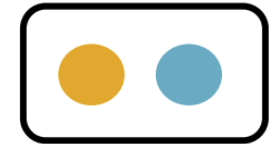
Neighbor
aggregation

Equivalent



Multi-set function

Examples of
multi-sets





Neighborhood aggregation with GCN and GraphSAGE

- Graph convolutional network (GCN)
 - Uses **element-wise** mean pooling over neighboring node embeddings

$$\text{mean}(\{\mathbf{x}_u\}_{u \in N(v)})$$

- GraphSAGE with max pooling
 - Uses **element-wise** max pooling over neighboring node embeddings

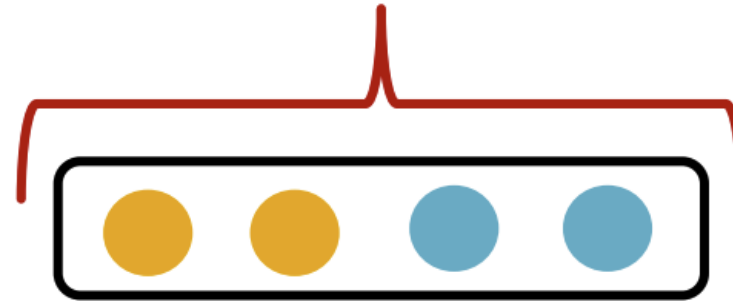
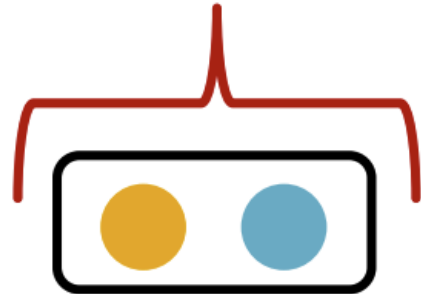
$$\text{max}(\{\mathbf{x}_u\}_{u \in N(v)})$$



GCN with mean pooling neighbor aggregation

- Take element-wise mean followed by linear layer and ReLU activation, $\max(0, x)$
- **Theorem:** GCN's aggregation function **cannot distinguish** different multi-sets with the same color (neighbor node) proportion

Failure case





GCN with mean pooling neighbor aggregation

- Assume node features (colors) are represented by one-hot encodings
- Example: Two distinct colors



$$= \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$



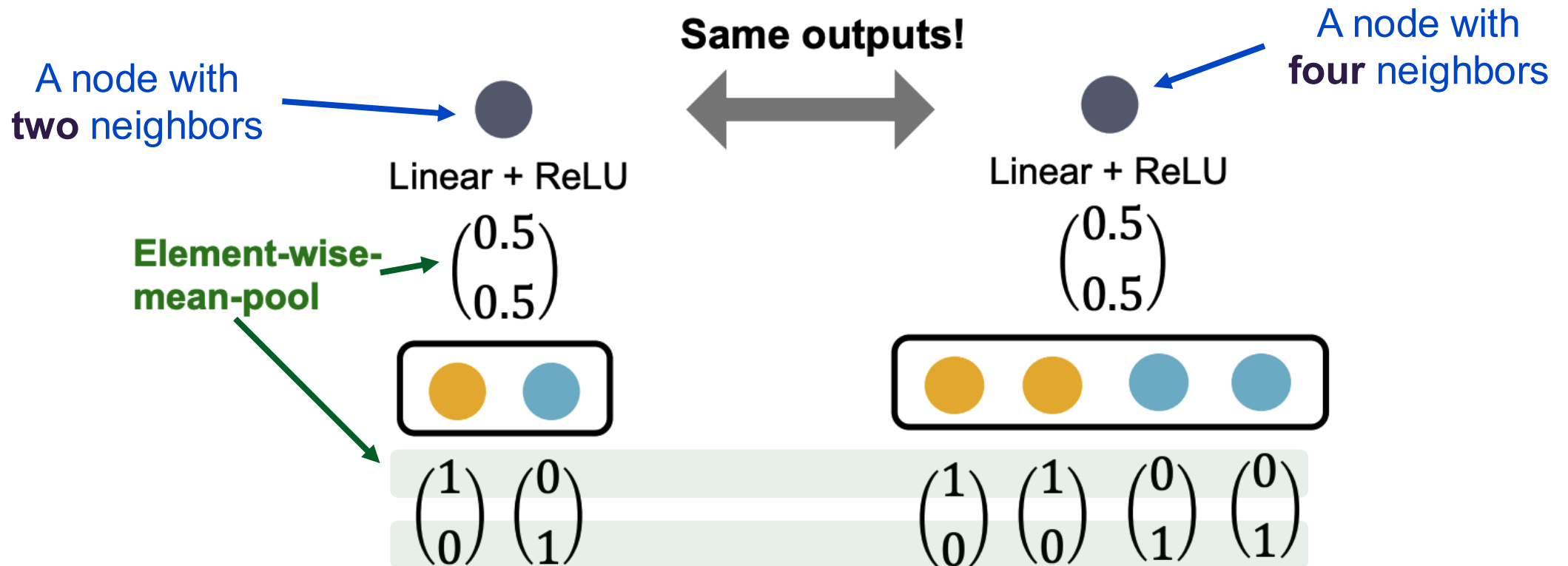
$$= \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- This will be sufficient to illustrate how a GCN fails for multisets with same color (neighbor node) proportions



GCN with mean pooling neighbor aggregation

- GCN with mean pooling **fails to distinguish** these multisets

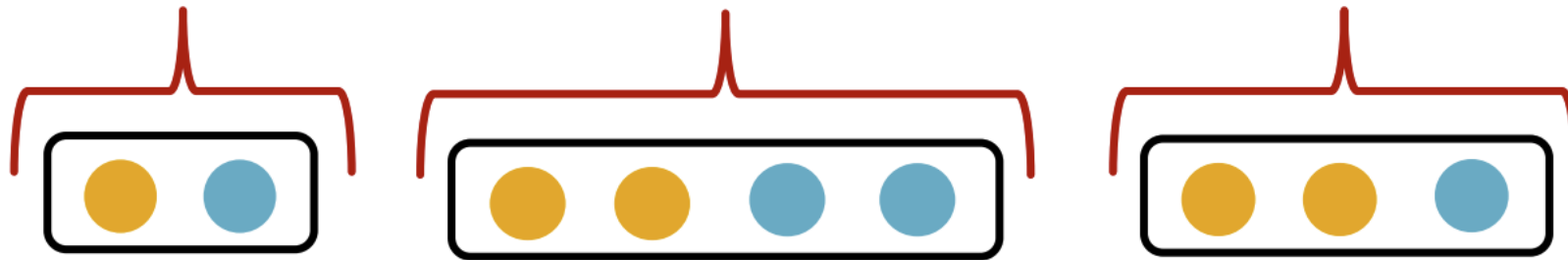




GraphSAGE with max pooling neighbor aggregation

- GraphSAGE applies an MLP and then applies pooling (usually max)
- Theorem:** GraphSAGE with max pooling aggregation cannot distinguish multi-sets with same set of distinct colors

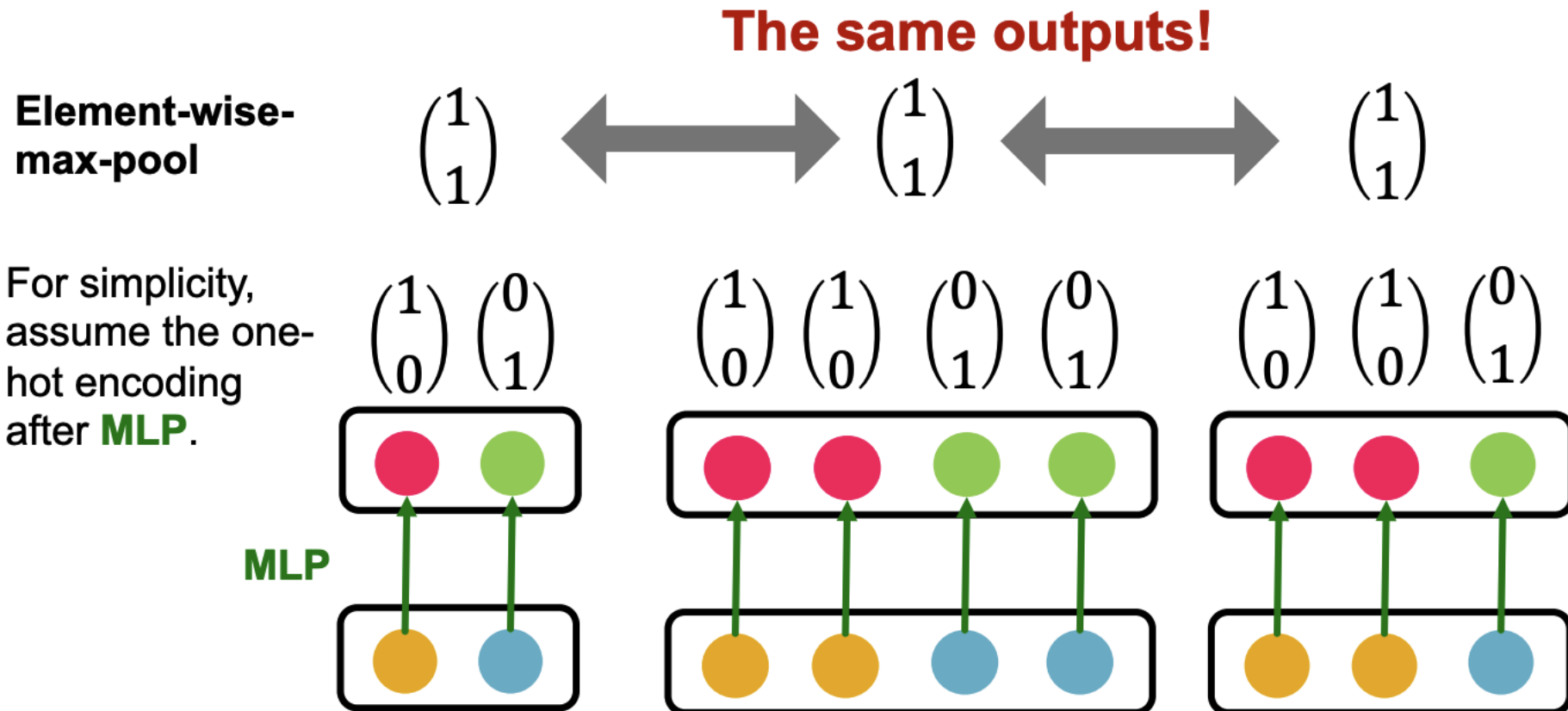
Failure case





GraphSAGE with max pooling neighbor aggregation

- GraphSAGE with max pooling **fails to distinguish** these multisets





Summary so far

- Expressive power of GNNs can be characterized by analysis of the neighbor aggregation function
- Neighbor aggregation viewed as a function over multi-sets
- GCN and GraphSAGE fail to distinguish certain multi-sets; hence they're **not injective**
- **Is there a more expressive GNN?**



Designing an injective GNN





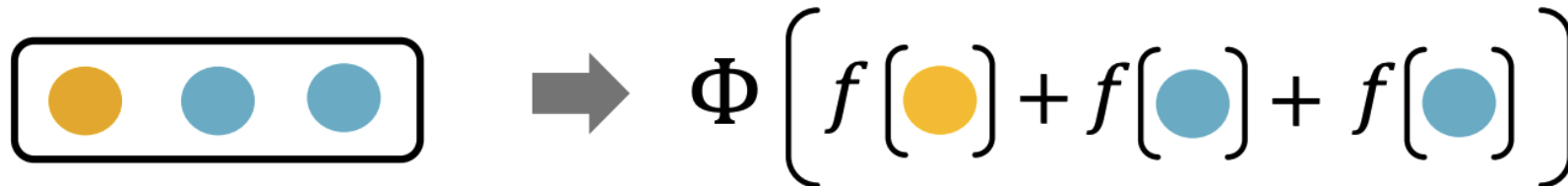
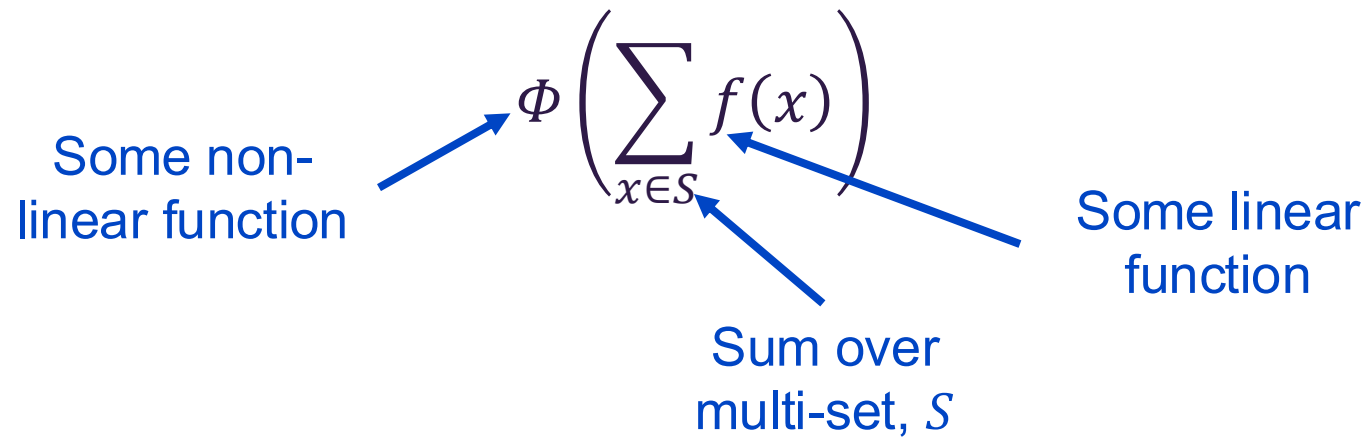
Designing the most expressive GNN

- Goal: Design a maximally expressive message passing GNN
- Can be achieved by designing an injective neighbor aggregation function over multi-sets
- We will design a neural network that achieves this



Injective multi-set function

Theorem 1: Any injective multi-set function can be expressed as a non-linear function of the summation of a linear function over the elements of the multi-set





Injective multi-set function

Proof intuition: f yields one-hot encodings of colors. Element wise summation of the encodings retains all the information about the input multi-set

$$\Phi \left(\sum_{x \in S} f(x) \right)$$

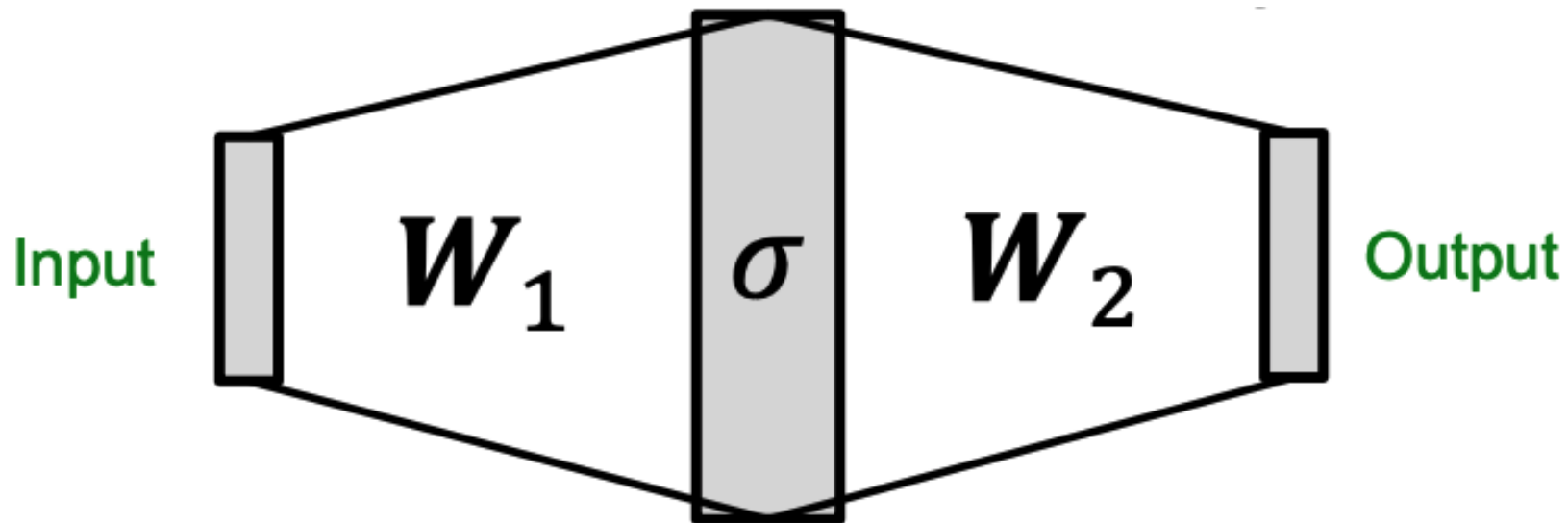
Example: $\Phi \left[\underbrace{f(\text{yellow})}_{\begin{pmatrix} 1 \\ 0 \end{pmatrix}} + \underbrace{f(\text{blue})}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}} + \underbrace{f(\text{blue})}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}} \right]$

One-hot $\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$



Universal approximation theorem

- How should we model Φ and f ?
- The *universal approximation theorem (UAT)* suggests we should use a multi-layer perceptron (MLP)
 - UAT: 1-hidden layer **MLP** with a sufficiently large hidden layer and an appropriate nonlinearity, σ , **can approximate any continuous function to arbitrary accuracy**





MLP as an injective multi-set function

- We can form a neural network that theoretically can model any injective multiset function

$$\Phi \left(\sum_{x \in S} f(x) \right) \cong MLP_{\Phi} \left(\sum_{x \in S} MLP_f(x) \right)$$

Theorem 1: All injective functions can be expressed in this form

UAT: f and Φ can be approximated to any accuracy with an MLP

- In practice, MLPs with 100-500 hidden units are typically sufficient



Graph isomorphism network (GIN)

- GIN node neighbor aggregation:
 - Apply an MLP to each node embedding,
 - element-wise sum

$$MLP_{\Phi} \left(\sum_{x \in S} MLP_f(x) \right)$$

GIN node
aggregation

- Theorem: GIN's neighbor aggregation function is injective
- How do we design the update function, MLP_{Φ} ?



Weisfeiler-Lehman (WL) Graph Kernel

- We will consider the color refinement test for graph isomorphism based on WL Kernel
 - WL test is a heuristic for graph isomorphism that compares graphs based on their local neighborhood structures.
 - Caveat: It cannot distinguish certain non-isomorphic graphs (we'll come back to this)



Weisfeiler-Lehman (WL) Graph Kernel

- WL color refinement test for isomorphism
- Given a graph, G , with a set of nodes V :
 - Assign an initial color $c^{(0)}(v)$ to each node $v \in V$
 - Iteratively refine node colors according to

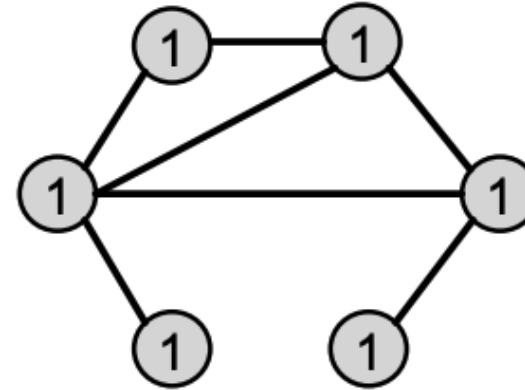
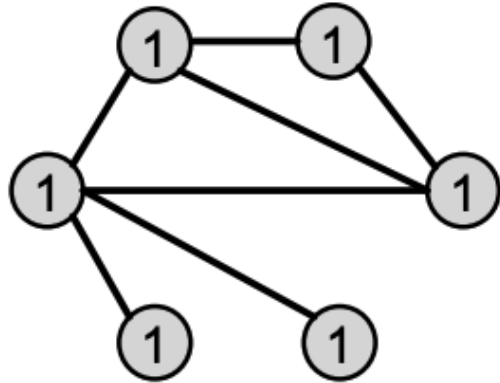
$$c^{(k+1)}(v) = \text{HASH} \left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)} \right)$$

where *HASH* maps different inputs to different colors

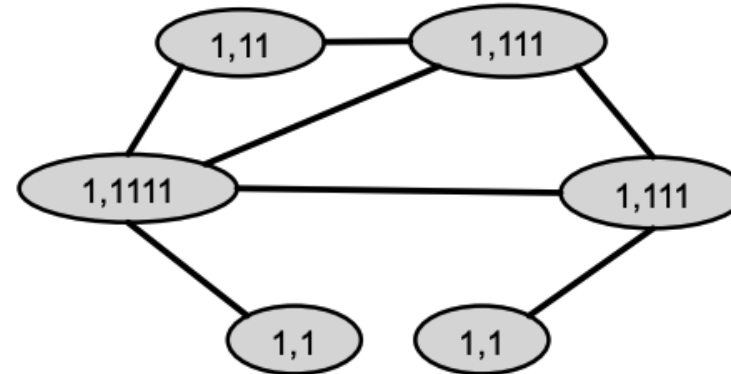
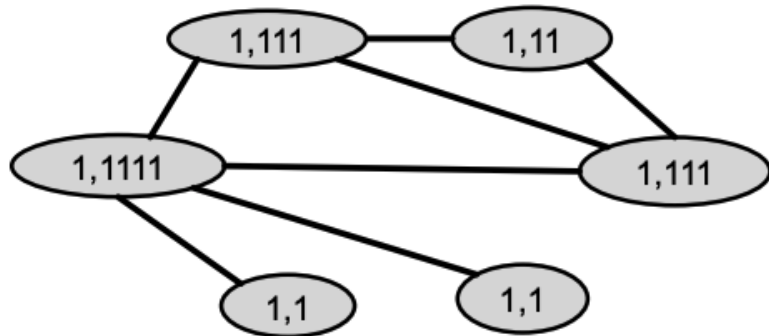
- After K steps, $c^{(K)}(v)$ summarizes the structure of v 's K -hop neighborhood

Color refinement on two graphs

- Assign the initial colors

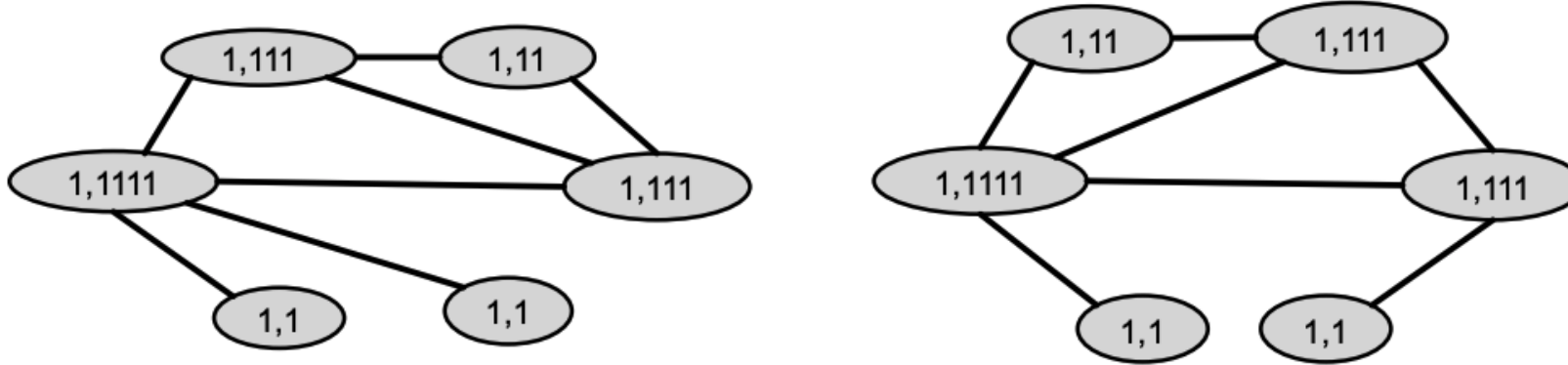


- Aggregate neighboring colors

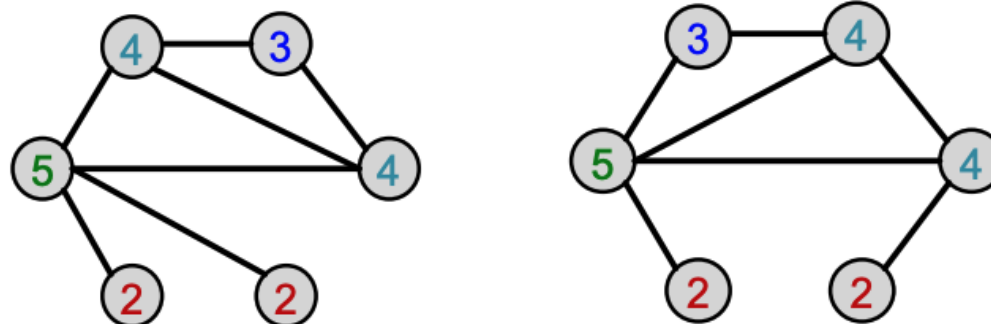


Color refinement on two graphs

- Aggregate neighboring colors



- Injectively** HASH the aggregated colors (i.e., make sure each distinct multi-set maps to a distinct color)



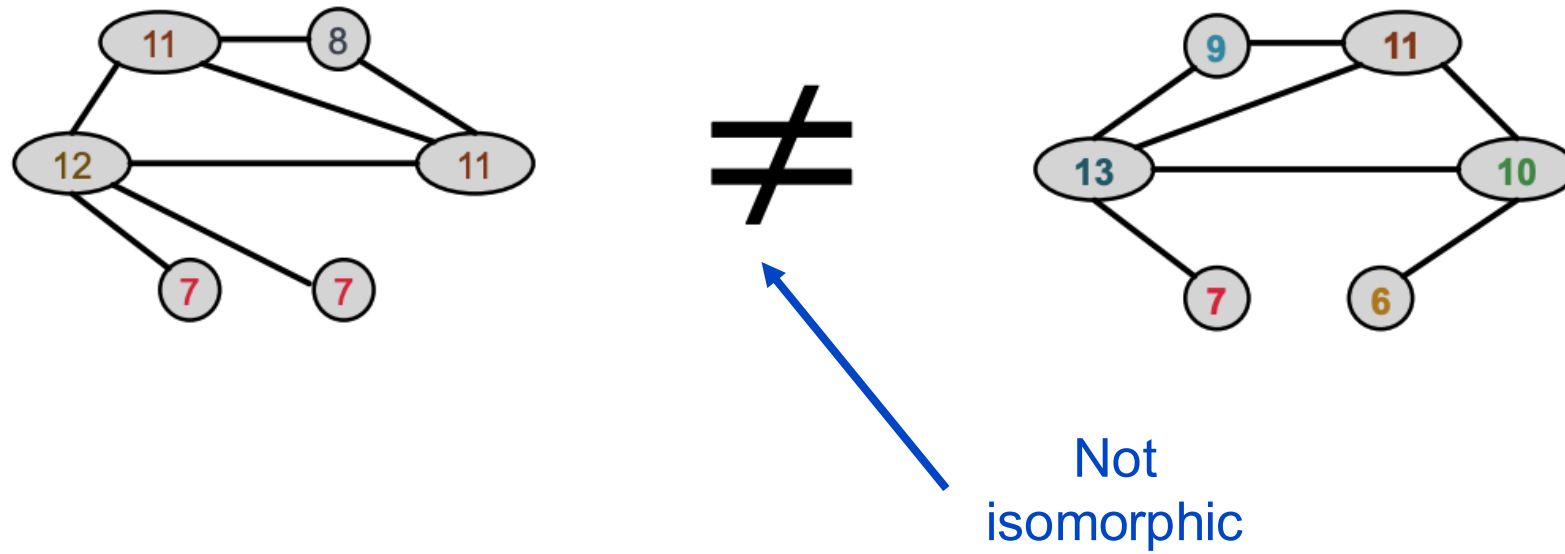
HASH table: Injective!

1,1	-->	2
1,11	-->	3
1,111	-->	4
1,1111	-->	5



Color refinement on two graphs

- Iterate until a stable coloring is reached – no two nodes that previously had different colors get assigned the same color, and no nodes with the same color get split
- Two graphs are considered **isomorphic** if they have the same color set (though false positives happen with this test)



The GIN model update function

- GIN uses a neural network to model the injective HASH function

$$c^{(k+1)}(v) = \text{HASH} \left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)} \right)$$

- Theorem:** Any injective function over the tuple

$$c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}$$

can be modeled as

$$MLP_{\Phi} \left[(1 + \varepsilon) MLP_f \left(c^{(k)}(v) \right) + \sum_{u \in N(v)} MLP_f \left(c^{(k)}(u) \right) \right]$$

where ε is a learnable scalar

Node
neighbor
embeddings

Node
embedding



The GIN model update function

- If the the input feature $c^{(0)}(v)$ is a one-hot, element-wise summation is injective

Example: $\Phi \left[\underbrace{\text{yellow circle}}_{\begin{pmatrix} 1 \\ 0 \end{pmatrix}} + \underbrace{\text{blue circle}}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}} + \underbrace{\text{blue circle}}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}} \right]$

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

- We only need to make sure Φ is injective

$$GINConv\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right) = \boxed{MLP_{\Phi}} \left[(1 + \varepsilon)MLP_f\left(c^{(k)}(v)\right) + \sum_{u \in N(v)} MLP_f\left(c^{(k)}(u)\right) \right]$$

Train this MLP to
generate a “one-hot” as
input to the next layer

GIN's node embedding updates

- Given a graph, G , with a set of nodes V :
 - Assign an **initial vector** $c^{(0)}(v)$ to each node $v \in V$ - $c^{(0)}(v)$ is the feature vector
 - Iteratively update node vectors according to

$$c^{(k+1)}(v) = GINConv \left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)} \right)$$

where $GINConv$ maps inputs to different embeddings

- After K steps, $c^{(K)}(v)$ summarizes the structure of v 's K -hop neighborhood



GIN and the Weisfeiler-Lehman graph kernel

- GIN can be understood as a differentiable neural version of the WL graph Kernel

	Update target	Update function
WL Graph Kernel	Node colors (one-hot)	HASH
GIN	Node embeddings (low-dim vectors)	GINConv

- Advantages of GIN over the WL graph kernel:
 - Node embeddings are low-dimensional; they can capture fine-grained similarity of different nodes (e.g., by cosine similarity)
 - Parameters of the update function can be learned for downstream tasks (i.e., node embeddings are useful in downstream tasks)



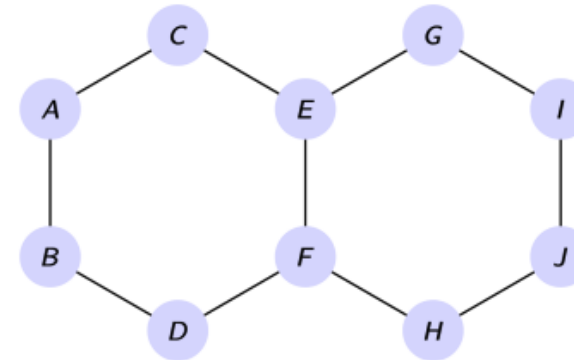
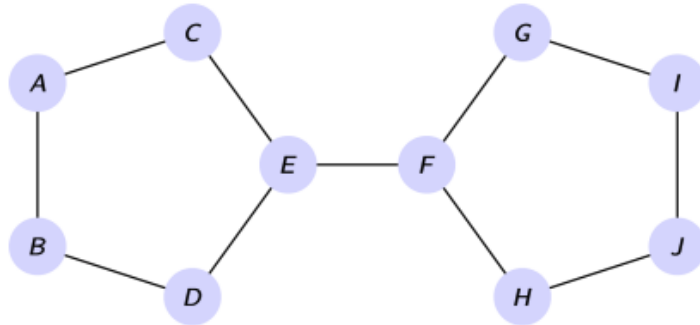
Expressive power of GIN

- The equivalence of GIN and WL graph kernel indicates they have the same expressiveness
 - Two graphs can be distinguished by GIN if and only if they can be distinguished by the WL kernel
- How powerful is this?
 - WL kernel has been theoretically proven and empirically shown to **distinguish most real-world graphs**
 - **But, sadly, not all**



GIN is not a panacea

- The WL kernel (and hence GIN) cannot distinguish between some important graph structures

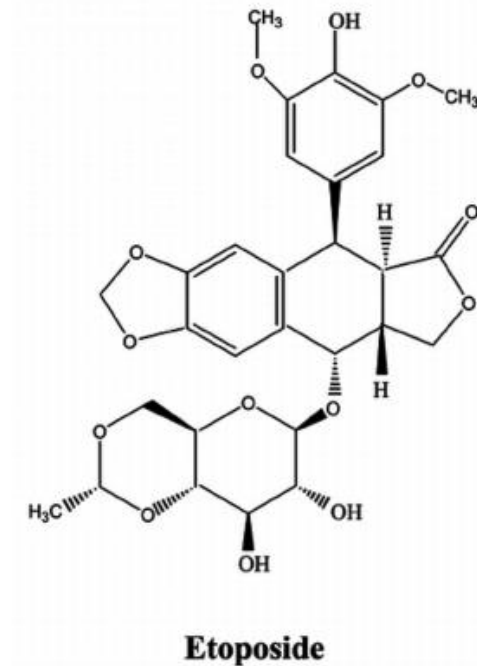
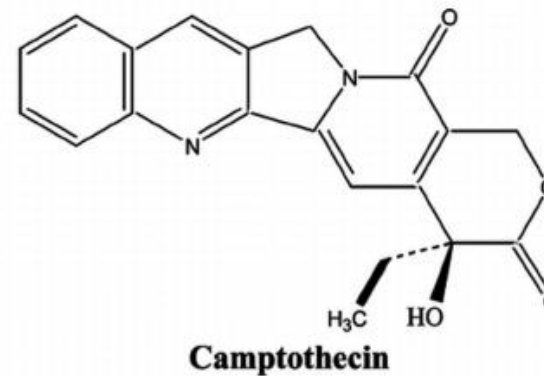
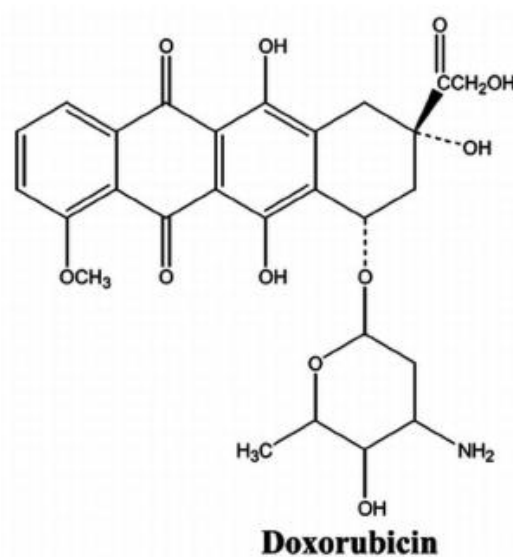


- It also cannot count important sub-structures such as cycles and cliques



Are the limitations of GINs important?

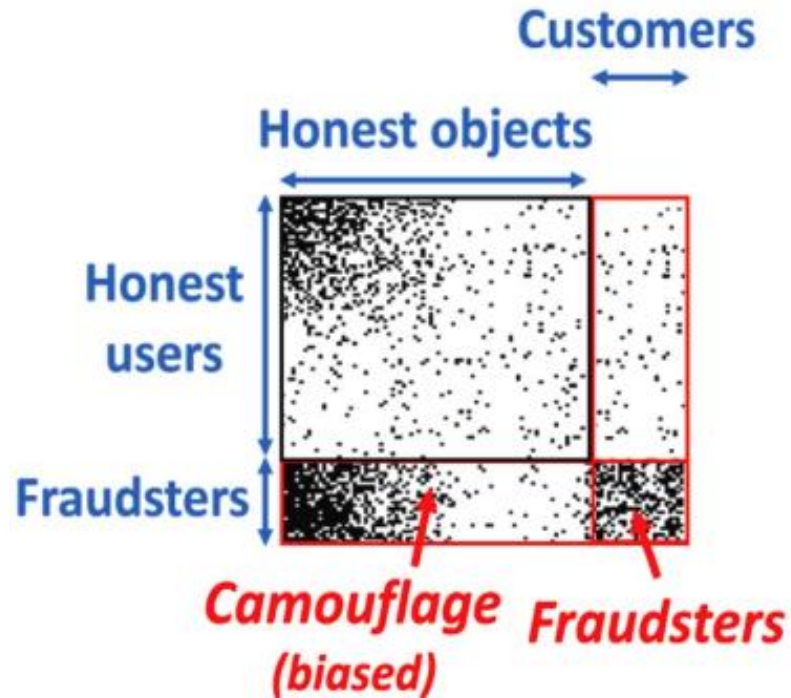
- Consider the inability to count **cycles**
- The bonds between atoms in chemical compounds often form cycles
- For example, hexagons are very common in organic compounds



Are the limitations of GINs important?

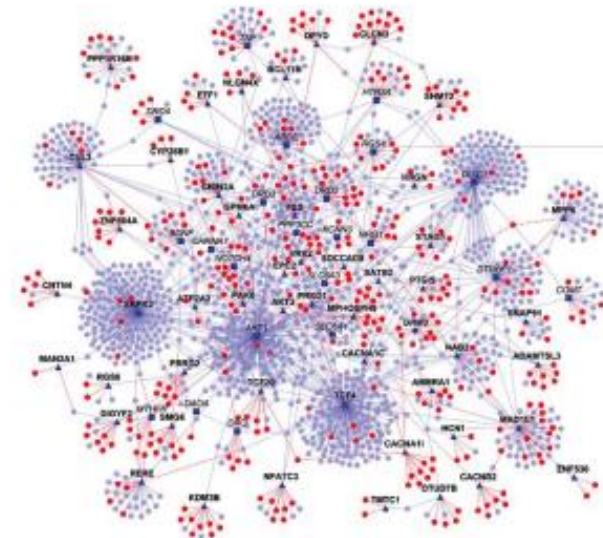
- Consider the inability to count **cliques**
- Cliques appear in many important application networks

Fraud detection in e-commerce networks



Hooi et al. 2016

Protein complexes in PPI networks

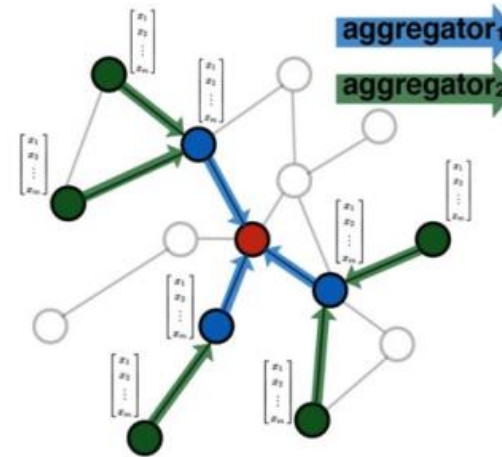
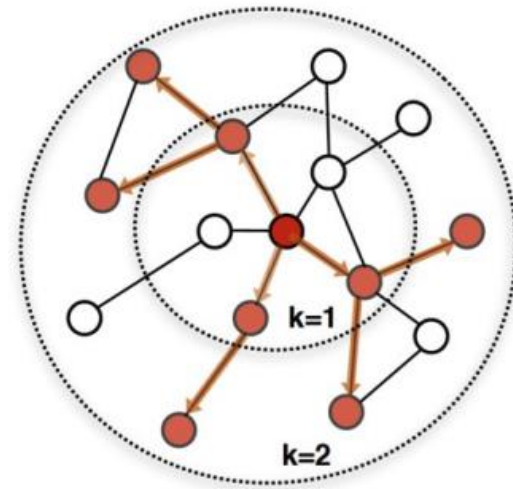


Vlaic et al. 2017



What should a “perfect” GNN do?

- A k -layer GNN embeds a node based on the k -hop neighborhood structure

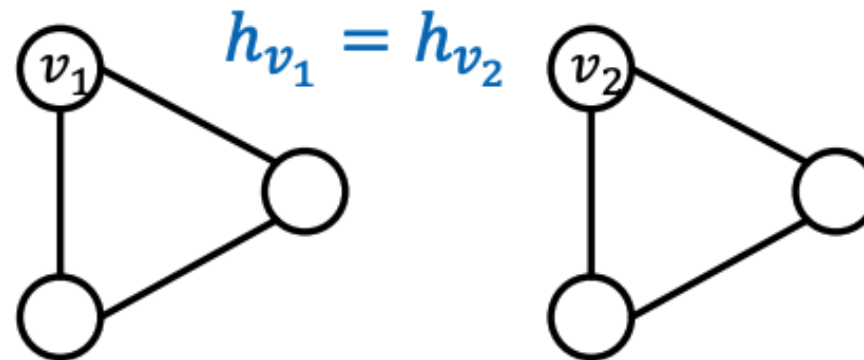


- A perfect GNN should build an **injective function** between **neighborhood structure** (regardless of hops) and **node embeddings**

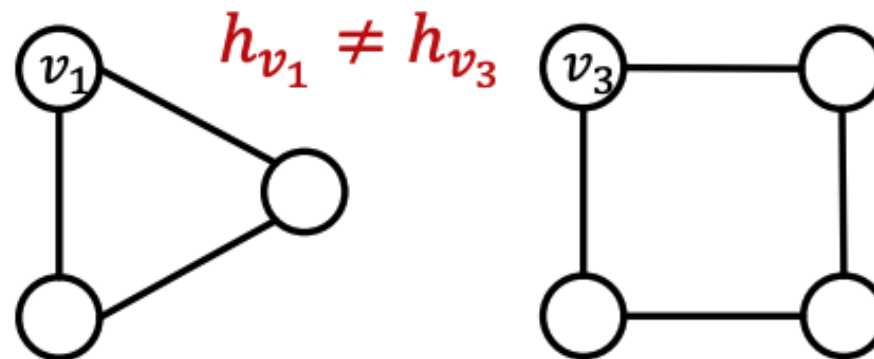


What should a “perfect” GNN do?

- Ignoring node attributes, if two nodes have the same neighborhood structure, they should have the same embedding



- If two nodes have different neighborhood structures, they should have different embeddings





Imperfections of existing GNNs

- We have already seen that GINs (our most expressive message passing GNN) fails to satisfy the second condition
- Additionally, sometimes we want the first condition to be violated based on their position in the graph – position-aware tasks such as street maps
- There are methods that are more expressive than WL kernel that start to address these limitations
 - Structurally-aware GNNs (e.g., see [Kanatsoulis 2024](#))
 - Position-aware GNNs (e.g., see [You 2019](#))

